Compte rendu

TP 2 — Programmation en C

du premier programme à l'outil système sûr

Module	Programmation système et réseaux sous Linux — travaux pratiques
Travaux	TP 2 — Programmation en C : du premier programme à l'outil système sûr
Binôme	ABDOUL Sajith / JOUVE Killian
Groupe	9
Date	21/10/2026
Formateur	Sofiane BOULJARF
Machines	ubuntu24-lab (Ubuntu 24.04.3 LTS, noyau 6.8.0-147-generic) et rhel9-lab (RHEL 9.8)
Compilateur	gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
Options	-Wall -Wextra -Werror -std=c17 -g

Sommaire

Préambule . . . 3

Partie 0 — Outillage . . . 3

Partie A — Fondamentaux du langage . . . 4

A1 — Premier programme, affichage et code de sortie . . . 4

A2 — Arguments, validation des entrées et boucles . . . 5

A3 — Fonctions, fichiers d'en-tête et Makefile . . . 8

A4 — Tableaux, chaînes et respect des bornes . . . 12

Partie B — Mémoire et sécurité . . . 16

B1 — Pointeurs et allocation dynamique . . . 16

B2 — Débordement de tampon : du programme vulnérable au programme sûr . . . 19

Partie C — Fichiers et appels système . . . 23

C1 — Fichiers tamponnés et comptage . . . 23

C2 — Appels système bruts : reprendre le programme du CM 1 . . . 25

C3 — Métadonnées, permissions et signaux d'alerte . . . 32

Partie D — Processus et signaux . . . 37

D1 — Lancer un programme et récupérer son résultat . . . 37

D2 — Arrêt propre sur signal . . . 40

Partie E — Synthèse évaluée . . . 45

E1 — Synthèse évaluée : l'outil d'audit d'arborescence . . . 45

Annexe 1 — Journal de bord . . . 59

Annexe 2 — Grille d'auto-évaluation . . . 60

Préambule

On a fait les exercices tous les deux, chacun sur sa machine du TP 1 : Sajith sur ubuntu24-lab et Killian sur rhel9-lab. Les sorties montrées dans ce rapport viennent de ubuntu24-lab (Ubuntu 24.04.3 LTS, compte sajith), avec gcc 13.3.0, GNU Make 4.3, strace 6.8 et valgrind 3.22.0. Tous les exercices sont compilés avec les options demandées -Wall -Wextra -Werror -std=c17 -g, sans aucun avertissement.

Partie 0 — Outillage

```
sajith@ubuntu24-lab:~$ sudo apt update
sajith@ubuntu24-lab:~$ sudo apt install -y build-essential strace valgrind
sajith@ubuntu24-lab:~$ gcc --version | head -1
gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
sajith@ubuntu24-lab:~$ make --version | head -1
GNU Make 4.3
sajith@ubuntu24-lab:~$ strace -V | head -1
strace -- version 6.8
sajith@ubuntu24-lab:~$ valgrind --version
valgrind-3.22.0
sajith@ubuntu24-lab:~$ mkdir -p ~/tp2-c && cd ~/tp2-c
sajith@ubuntu24-lab:~$ echo 'int main(void){return 0;}' > essai.c
sajith@ubuntu24-lab:~$ gcc -Wall -Wextra -Werror -std=c17 -g -o essai essai.c
sajith@ubuntu24-lab:~$ ./essai ; echo "code de sortie : $?"
code de sortie : 0
```

Les quatre outils répondent. essai.c compile sans aucun avertissement avec les options imposées, et le programme rend le code 0.

Partie A — Fondamentaux du langage

A1 — Premier programme, affichage et code de sortie

Objectif. Compiler, exécuter et retourner un code de sortie exploitable par un script : succès, usage incorrect et argument inconnu ont chacun leur code.

Fichiers. a1_bonjour.c

Code source

a1_bonjour.c

```
sajith@ubuntu24-lab:~/tp2-c$ cat a1_bonjour.c
#include <stdio.h>
#include <string.h>

int main(int argc, char *argv[])
{
    if (argc == 1) {
        printf("outil=tp2-c version=1.0 compile=%s\n", __DATE__);
        return 0;
    }

    if (argc > 2) {
        fprintf(stderr, "usage : %s [--version]\n", argv[0]);
        return 1;
    }

    if (strcmp(argv[1], "--version") == 0) {
        printf("1.0\n");
        return 0;
    }

    fprintf(stderr, "argument inconnu : %s\n", argv[1]);
    fprintf(stderr, "usage : %s [--version]\n", argv[0]);
    return 2;
}
```

Compilation

```
sajith@ubuntu24-lab:~/tp2-c$ gcc -Wall -Wextra -Werror -std=c17 -g -o a1_bonjour a1_bonjour.c
[aucune sortie = aucun avertissement]
```

Sorties obtenues

```
sajith@ubuntu24-lab:~/tp2-c$ ./a1_bonjour ; echo "code=$?"
outil=tp2-c version=1.0 compile=Oct 19 2026
code=0

sajith@ubuntu24-lab:~/tp2-c$ ./a1_bonjour --version ; echo "code=$?"
1.0
code=0

sajith@ubuntu24-lab:~/tp2-c$ ./a1_bonjour a b ; echo "code=$?"          [test d'erreur volontaire]
usage : ./a1_bonjour [--version]
code=1

sajith@ubuntu24-lab:~/tp2-c$ ./a1_bonjour --inconnu ; echo "code=$?"     [test d'erreur
↪ volontaire]
argument inconnu : --inconnu
usage : ./a1_bonjour [--version]
code=2

sajith@ubuntu24-lab:~/tp2-c$ ./a1_bonjour --inconnu > sortie.txt ; echo "code=$?"
argument inconnu : --inconnu
usage : ./a1_bonjour [--version]
code=2
sajith@ubuntu24-lab:~/tp2-c$ wc -c sortie.txt
0 sortie.txt
```

Réponses aux questions

Question 1.1 — Pourquoi un diagnostic doit-il être écrit sur la sortie d'erreur et non sur la sortie standard ? Reliez votre réponse à la redirection des flux vue au TP 1.

La sortie standard, c'est le résultat du programme ; la sortie d'erreur, c'est pour les messages de problème. Ce sont deux « tuyaux » différents. Quand on fait une redirection avec >, seul le tuyau du résultat part dans le fichier, les erreurs restent à l'écran. Si mon message d'erreur partait sur la sortie standard, il se retrouverait mélangé aux résultats, par exemple dans un fichier de preuves. Mon dernier test le montre : avec ./a1_bonjour --inconnu > sortie.txt, le message s'affiche quand même et sortie.txt fait 0 octet. Au TP 1, j'ai justement utilisé 2>&1 pour récupérer aussi les erreurs dans mon fichier de preuves, parce que sans ça elles n'y étaient pas.

Question 1.2 — Quelle macro le compilateur fournit-il pour la date et l'heure de compilation ? Pourquoi est-ce préférable à une date écrite dans le code ?

C'est __DATE__ pour la date et __TIME__ pour l'heure. Le compilateur les remplace tout seul au moment de la compilation. Si j'écrivais la date à la main, je devrais penser à la changer à chaque recompilation, et elle finirait forcément par être fausse. Avec __DATE__, la date affichée correspond toujours au programme qui tourne vraiment, donc on sait quelle version est installée sur la machine.

Difficultés rencontrées

Le premier essai rendait bien le code 2 pour un argument inconnu, mais sans afficher aucun message : l'utilisateur ne savait pas pourquoi le programme refusait. J'ai ajouté les messages sur la sortie d'erreur avec fprintf(stderr, ...) et le cas « trop d'arguments » avec son propre code de sortie (1).

A2 — Arguments, validation des entrées et boucles

Objectif. Valider avant d'agir : convertir un nombre d'octets avec strtol, refuser chaque entrée invalide avec un message et un code de sortie distinct par cause.

Fichiers. a2_convertisseur.c

Choix documenté. Capacité maximale acceptée : 1 099 511 627 776 octets (1 To), fixée par la constante MAX_OCTETS. Au-delà, ou si strtol signale un dépassement (errno = ERANGE), la valeur est refusée avec le code 5.

Code source

a2_convertisseur.c

```
sajith@ubuntu24-lab:~/tp2-c$ cat a2_convertisseur.c
#include <stdio.h>
#include <stdlib.h>
#include <errno.h>

#define MAX_OCTETS 1099511627776L

int main(int argc, char *argv[])
{
    char *fin;
    long valeur;

    if (argc < 2) {
        fprintf(stderr, "erreur : aucun argument\nusage : %s <octets>\n", argv[0]);
        return 1;
    }
    if (argc > 2) {
        fprintf(stderr, "erreur : trop d'arguments\nusage : %s <octets>\n", argv[0]);
        return 2;
    }

    errno = 0;
    valeur = strtol(argv[1], &fin, 10);

    if (fin == argv[1] || *fin != '\0') {
        fprintf(stderr, "erreur : argument non numerique : %s\n", argv[1]);
        return 3;
    }
    if (valeur < 0) {
        fprintf(stderr, "erreur : valeur negative : %ld\n", valeur);
        return 4;
    }
    if (errno == ERANGE || valeur > MAX_OCTETS) {
        fprintf(stderr, "erreur : valeur trop grande (maximum %ld octets)\n", MAX_OCTETS);
        return 5;
    }

    const char *unites[] = {"octets", "Ko", "Mo", "Go", "To"};
    double taille = (double)valeur;
    int i = 0;

    while (taille >= 1024.0 && i < 4) {
        taille = taille / 1024.0;
        i++;
    }
```

```
    printf("%ld octets = %.2f %s\n", valeur, taille, unites[i]);
    return 0;
}
```

Compilation

```
sajith@ubuntu24-lab:~/tp2-c$ gcc -Wall -Wextra -Werror -std=c17 -g -o a2_convertisseur
↪ a2_convertisseur.c
[aucune sortie = aucun avertissement]
```

Sorties obtenues

```
sajith@ubuntu24-lab:~/tp2-c$ ./a2_convertisseur 1536 ; echo "code=$?"
1536 octets = 1.50 Ko
code=0
sajith@ubuntu24-lab:~/tp2-c$ ./a2_convertisseur 3221225472 ; echo "code=$?"
3221225472 octets = 3.00 Go
code=0
sajith@ubuntu24-lab:~/tp2-c$ ./a2_convertisseur 0 ; echo "code=$?"
0 octets = 0.00 octets
code=0
sajith@ubuntu24-lab:~/tp2-c$ ./a2_convertisseur 512 ; echo "code=$?"
512 octets = 512.00 octets
code=0
sajith@ubuntu24-lab:~/tp2-c$ ./a2_convertisseur 1099511627776 ; echo "code=$?"
1099511627776 octets = 1.00 To
code=0
sajith@ubuntu24-lab:~/tp2-c$ ./a2_convertisseur ; echo "code=$?"                          [test
↪ d'erreur volontaire]
erreur : aucun argument
usage : ./a2_convertisseur <octets>
code=1
sajith@ubuntu24-lab:~/tp2-c$ ./a2_convertisseur 1 2 ; echo "code=$?"                     [test
↪ d'erreur volontaire]
erreur : trop d'arguments
usage : ./a2_convertisseur <octets>
code=2
sajith@ubuntu24-lab:~/tp2-c$ ./a2_convertisseur '' ; echo "code=$?"                      [test
↪ d'erreur volontaire]
erreur : argument non numerique :
code=3
sajith@ubuntu24-lab:~/tp2-c$ ./a2_convertisseur abc ; echo "code=$?"                     [test
↪ d'erreur volontaire]
erreur : argument non numerique : abc
code=3
sajith@ubuntu24-lab:~/tp2-c$ ./a2_convertisseur 12abc ; echo "code=$?"                   [test
↪ d'erreur volontaire]
erreur : argument non numerique : 12abc
code=3
sajith@ubuntu24-lab:~/tp2-c$ ./a2_convertisseur -5 ; echo "code=$?"                      [test
↪ d'erreur volontaire]
erreur : valeur negative : -5
code=4
sajith@ubuntu24-lab:~/tp2-c$ ./a2_convertisseur 99999999999999999999 ; echo "code=$?"   [test
↪ d'erreur volontaire]
erreur : valeur trop grande (maximum 1099511627776 octets)
code=5
```

Tableau récapitulatif des tests

Commande	Sortie	Code
./a2_convertisseur 1536	1536 octets = 1.50 Ko	0
./a2_convertisseur 3221225472	3221225472 octets = 3.00 Go	0
./a2_convertisseur 0	0 octets = 0.00 octets (cas limite valide)	0
./a2_convertisseur 512	512 octets = 512.00 octets	0
./a2_convertisseur 1099511627776	1099511627776 octets = 1.00 To (maximum accepté)	0
./a2_convertisseur	erreur : aucun argument	1
./a2_convertisseur 1 2	erreur : trop d'arguments	2
./a2_convertisseur ''	erreur : argument non numerique	3
./a2_convertisseur abc	erreur : argument non numerique : abc	3
./a2_convertisseur 12abc	erreur : argument non numerique : 12abc	3
./a2_convertisseur -5	erreur : valeur negative : -5	4
./a2_convertisseur 99999999999999999999	erreur : valeur trop grande	5

Réponses aux questions

Question 2.1 — Montrez, par un exemple exécuté, ce que rend atoi("abc") et pourquoi cela interdit toute validation.

J'ai fait un petit programme de test (q2_test.c) : atoi("abc") rend 0, et atoi("0") rend aussi 0. Le programme ne peut donc pas savoir si l'utilisateur a vraiment tapé 0 ou n'importe quoi, et atoi ne signale aucune erreur. Avec strtol, on a un pointeur qui indique où la lecture s'est arrêtée : s'il reste des caractères (« 12abc ») ou si rien n'a été lu (« abc », chaîne vide), je sais que l'entrée est invalide et je la refuse.

Question 2.2 — Que se passe-t-il si l'on convertit -1 en unsigned long ? Donnez la valeur obtenue et expliquez le lien avec les débordements de mémoire.

Mon test donne (unsigned long)-1 = 18446744073709551615. Un type non signé ne peut pas être négatif, donc -1 « repasse par le haut » et devient la plus grande valeur possible sur 64 bits, un peu comme un compteur qui reculerait sous zéro. Le danger : si cette valeur sert de taille pour une copie ou une allocation, le programme croit devoir traiter des milliards de milliards d'octets et écrit partout en mémoire. C'est pour ça que mon convertisseur refuse les valeurs négatives (code 4) avant de faire quoi que ce soit.

Difficultés rencontrées

Aucun avertissement de compilation. Point d'attention : le seul test *fin != '\0' laisse passer la chaîne vide, car strtol ne lit rien et le pointeur de fin est déjà sur le caractère de fin. J'ai ajouté le test fin == argv[1] pour refuser aussi ce cas (vérifié avec ./a2_convertisseur '').

A3 — Fonctions, fichiers d'en-tête et Makefile

Objectif. Découper le programme en modules (en-tête, implémentation, programme principal) et le reconstruire avec un Makefile.

Fichiers. a3_conversion.h, a3_conversion.c, a3_main.c, Makefile

Choix documenté. Les codes de sortie et la capacité maximale (1 To) sont identiques à A2 : le comportement est intégralement conservé. Les fonctions de a3_conversion.c n'affichent rien ; tout l'affichage est dans a3_main.c.

Code source

a3_conversion.h

```
sajith@ubuntu24-lab:~/tp2-c$ cat a3_conversion.h
#ifndef A3_CONVERSION_H
#define A3_CONVERSION_H

#define MAX_OCTETS 1099511627776L

enum {
    CONV_OK = 0,
    CONV_NON_NUMERIQUE = 3,
    CONV_NEGATIF = 4,
    CONV_TROP_GRAND = 5
};

int convertir_octets(const char *texte, long *valeur);
void unite_lisible(long octets, double *taille, const char **unite);

#endif
```

a3_conversion.c

```
sajith@ubuntu24-lab:~/tp2-c$ cat a3_conversion.c
#include <stdlib.h>
#include <errno.h>
#include "a3_conversion.h"

int convertir_octets(const char *texte, long *valeur)
{
    char *fin;
    long v;

    errno = 0;
    v = strtol(texte, &fin, 10);

    if (fin == texte || *fin != '\0') {
        return CONV_NON_NUMERIQUE;
    }
    if (v < 0) {
        return CONV_NEGATIF;
    }
    if (errno == ERANGE || v > MAX_OCTETS) {
        return CONV_TROP_GRAND;
    }

    *valeur = v;
    return CONV_OK;
}

void unite_lisible(long octets, double *taille, const char **unite)
{
    static const char *unites[] = {"octets", "Ko", "Mo", "Go", "To"};
    double t = (double)octets;
    int i = 0;

    while (t >= 1024.0 && i < 4) {
        t = t / 1024.0;
        i++;
    }

    *taille = t;
    *unite = unites[i];
}
```

a3_main.c

```
sajith@ubuntu24-lab:~/tp2-c$ cat a3_main.c
#include <stdio.h>
#include "a3_conversion.h"

int main(int argc, char *argv[])
{
    long valeur;
    double taille;
    const char *unite;
    int code;

    if (argc < 2) {
        fprintf(stderr, "erreur : aucun argument\nusage : %s <octets>\n", argv[0]);
        return 1;
    }
    if (argc > 2) {
        fprintf(stderr, "erreur : trop d'arguments\nusage : %s <octets>\n", argv[0]);
        return 2;
    }

    code = convertir_octets(argv[1], &valeur);

    if (code == CONV_NON_NUMERIQUE) {
        fprintf(stderr, "erreur : argument non numerique : %s\n", argv[1]);
        return code;
    }
    if (code == CONV_NEGATIF) {
        fprintf(stderr, "erreur : valeur negative : %s\n", argv[1]);
        return code;
    }
    if (code == CONV_TROP_GRAND) {
        fprintf(stderr, "erreur : valeur trop grande (maximum %ld octets)\n", MAX_OCTETS);
        return code;
    }

    unite_lisible(valeur, &taille, &unite);
    printf("%ld octets = %.2f %s\n", valeur, taille, unite);
    return 0;
}
```

Makefile

```
sajith@ubuntu24-lab:~/tp2-c$ cat Makefile
CC = gcc
CFLAGS = -Wall -Wextra -Werror -std=c17 -g
CIBLE = a3_convertisseur
OBJETS = a3_main.o a3_conversion.o

all: $(CIBLE)

$(CIBLE): $(OBJETS)
    $(CC) $(CFLAGS) -o $(CIBLE) $(OBJETS)

a3_main.o: a3_main.c a3_conversion.h
    $(CC) $(CFLAGS) -c a3_main.c

a3_conversion.o: a3_conversion.c a3_conversion.h
    $(CC) $(CFLAGS) -c a3_conversion.c

clean:
    rm -f $(OBJETS) $(CIBLE)

.PHONY: all clean
```

Compilation

```
sajith@ubuntu24-lab:~/tp2-c$ make clean
rm -f a3_main.o a3_conversion.o a3_convertisseur

sajith@ubuntu24-lab:~/tp2-c$ make                                    [1re construction]
gcc -Wall -Wextra -Werror -std=c17 -g -c a3_main.c
gcc -Wall -Wextra -Werror -std=c17 -g -c a3_conversion.c
gcc -Wall -Wextra -Werror -std=c17 -g -o a3_convertisseur a3_main.o a3_conversion.o

sajith@ubuntu24-lab:~/tp2-c$ make                                    [2e exécution, sans
↪ modification]
make: Nothing to be done for 'all'.

sajith@ubuntu24-lab:~/tp2-c$ touch a3_main.c                         [modification de a3_main.c
↪ seulement]
sajith@ubuntu24-lab:~/tp2-c$ make
gcc -Wall -Wextra -Werror -std=c17 -g -c a3_main.c
gcc -Wall -Wextra -Werror -std=c17 -g -o a3_convertisseur a3_main.o a3_conversion.o
```

Sorties obtenues

```
sajith@ubuntu24-lab:~/tp2-c$ ./a3_convertisseur 1536 ; echo "code=$?"
1536 octets = 1.50 Ko
code=0
sajith@ubuntu24-lab:~/tp2-c$ ./a3_convertisseur 3221225472 ; echo "code=$?"
3221225472 octets = 3.00 Go
code=0
sajith@ubuntu24-lab:~/tp2-c$ ./a3_convertisseur 0 ; echo "code=$?"
0 octets = 0.00 octets
code=0
sajith@ubuntu24-lab:~/tp2-c$ ./a3_convertisseur '' ; echo "code=$?"                     [test
↪ d'erreur volontaire]
erreur : argument non numerique :
code=3
sajith@ubuntu24-lab:~/tp2-c$ ./a3_convertisseur abc ; echo "code=$?"                    [test
↪ d'erreur volontaire]
erreur : argument non numerique : abc
code=3
sajith@ubuntu24-lab:~/tp2-c$ ./a3_convertisseur 12abc ; echo "code=$?"                  [test
↪ d'erreur volontaire]
erreur : argument non numerique : 12abc
code=3
sajith@ubuntu24-lab:~/tp2-c$ ./a3_convertisseur -5 ; echo "code=$?"                     [test
↪ d'erreur volontaire]
erreur : valeur negative : -5
code=4
sajith@ubuntu24-lab:~/tp2-c$ ./a3_convertisseur 99999999999999999 ; echo "code=$?"   [test
↪ d'erreur volontaire]
erreur : valeur trop grande (maximum 1099511627776 octets)
code=5
sajith@ubuntu24-lab:~/tp2-c$ ./a3_convertisseur ; echo "code=$?"                        [test
↪ d'erreur volontaire]
erreur : aucun argument
usage : ./a3_convertisseur <octets>
code=1
sajith@ubuntu24-lab:~/tp2-c$ ./a3_convertisseur 1 2 ; echo "code=$?"                    [test
↪ d'erreur volontaire]
erreur : trop d'arguments
usage : ./a3_convertisseur <octets>
code=2
```

Réponses aux questions

Question 3.1 — À quoi sert la protection contre la double inclusion dans un fichier d'en-tête, et par quelles directives l'obtient-on ?

Un même fichier d'en-tête peut être inclus plusieurs fois, par exemple directement et par un autre en-tête. Sans protection, le compilateur verrait deux fois les mêmes déclarations (ici mon enum des codes d'erreur) et refuserait de compiler. On s'en protège avec #ifndef A3_CONVERSION_H et #define A3_CONVERSION_H au début, puis #endif à la fin. La première fois, le nom n'existe pas : il est défini et le contenu est lu. Les fois suivantes, le nom existe déjà, donc tout le contenu est sauté.

Question 3.2 — Pourquoi est-il préférable que les fonctions de conversion n'affichent rien ? Donnez un cas concret où l'affichage interne deviendrait gênant.

Je le vois comme dans un restaurant : la cuisine prépare les plats, mais c'est la salle qui parle aux clients. Mes fonctions de conversion rendent juste un résultat et un code d'erreur, et c'est main qui décide quoi afficher et où. Si la fonction affichait elle-même ses erreurs, elle imposerait son message partout où on la réutilise. Exemple concret : l'outil d'audit final appelle les fonctions des milliers de fois et écrit un journal ; des messages affichés directement par la fonction viendraient polluer l'écran et le journal, sans que l'outil puisse les contrôler.

Difficultés rencontrées

Les lignes de commande d'un Makefile doivent commencer par une tabulation et non par des espaces, sinon make échoue avec « missing separator ». Le copier-coller dans le terminal remplace les tabulations par des espaces ; j'ai donc remplacé les quatre espaces de début de ligne par une tabulation avec sed et vérifié les tabulations avec cat -A Makefile (caractère ^I).

A4 — Tableaux, chaînes et respect des bornes

Objectif. Ne jamais dépasser la capacité d'un tableau : contrôler chaque indice et refuser les données qui ne tiennent pas.

Fichiers. a4_statistiques.c

Choix documenté. Capacité : MAX_VALEURS = 16. L'indice d'écriture n est comparé à la capacité avant chaque écriture dans le tableau. Chaque valeur est validée par strtol et bornée à [INT_MIN, INT_MAX], ce qui garantit que la somme de 16 valeurs tient dans un long. Codes : 0 succès, 1 aucune valeur, 2 trop de valeurs, 3 non numérique, 4 hors bornes. Les deux versions de la moyenne sont obtenues à partir du même fichier : la version fausse est compilée avec -DMOYENNE_ENTIERE.

Code source

a4_statistiques.c

```
sajith@ubuntu24-lab:~/tp2-c$ cat a4_statistiques.c
#include <stdio.h>
#include <stdlib.h>
#include <errno.h>
#include <limits.h>

#define MAX_VALEURS 16

int main(int argc, char *argv[])
{
    long valeurs[MAX_VALEURS];
    int n = 0;

    if (argc < 2) {
        fprintf(stderr, "erreur : aucune valeur\nusage : %s <entier> [entier...]\n", argv[0]);
        return 1;
    }

    for (int i = 1; i < argc; i++) {
        char *fin;
        long v;

        errno = 0;
        v = strtol(argv[i], &fin, 10);

        if (fin == argv[i] || *fin != '\0') {
            fprintf(stderr, "erreur : valeur non numerique : %s\n", argv[i]);
            return 3;
        }
        if (errno == ERANGE || v < INT_MIN || v > INT_MAX) {
            fprintf(stderr, "erreur : valeur hors des bornes d'un entier : %s\n", argv[i]);
            return 4;
        }
        if (n >= MAX_VALEURS) {
            fprintf(stderr, "erreur : trop de valeurs (maximum %d)\n", MAX_VALEURS);
            return 2;
        }

        valeurs[n] = v;
        n++;
    }

    long somme = 0;
    long min = valeurs[0];
    long max = valeurs[0];
```

```
    for (int i = 0; i < n; i++) {
        somme = somme + valeurs[i];
        if (valeurs[i] < min) {
            min = valeurs[i];
        }
        if (valeurs[i] > max) {
            max = valeurs[i];
        }
    }

#ifdef MOYENNE_ENTIERE
    double moyenne = somme / n;
#else
    double moyenne = (double)somme / n;
#endif

    printf("elements=%d somme=%ld moyenne=%.2f min=%ld max=%ld\n", n, somme, moyenne, min, max);
    return 0;
}
```

Compilation

```
sajith@ubuntu24-lab:~/tp2-c$ gcc -Wall -Wextra -Werror -std=c17 -g -o a4_statistiques
↪ a4_statistiques.c
[aucune sortie = aucun avertissement]

Version volontairement fausse (division entière), même fichier source :
sajith@ubuntu24-lab:~/tp2-c$ gcc -Wall -Wextra -Werror -std=c17 -g -DMOYENNE_ENTIERE -o a4_faux
↪ a4_statistiques.c
[aucune sortie = aucun avertissement]
```

Sorties obtenues

```
sajith@ubuntu24-lab:~/tp2-c$ ./a4_statistiques 12 7 30 -4 ; echo "code=$?"
elements=4 somme=45 moyenne=11.25 min=-4 max=30
code=0
sajith@ubuntu24-lab:~/tp2-c$ ./a4_statistiques 5 5 5 5 ; echo "code=$?"
elements=4 somme=20 moyenne=5.00 min=5 max=5
code=0
sajith@ubuntu24-lab:~/tp2-c$ ./a4_statistiques -10 -20 -3 ; echo "code=$?"
elements=3 somme=-33 moyenne=-11.00 min=-20 max=-3
code=0
sajith@ubuntu24-lab:~/tp2-c$ ./a4_statistiques $(seq 1 16) ; echo "code=$?"           [16
↪ valeurs = capacité]
elements=16 somme=136 moyenne=8.50 min=1 max=16
code=0
sajith@ubuntu24-lab:~/tp2-c$ ./a4_statistiques $(seq 1 17) ; echo "code=$?"           [test
↪ d'erreur volontaire : 17 valeurs]
erreur : trop de valeurs (maximum 16)
code=2
sajith@ubuntu24-lab:~/tp2-c$ ./a4_statistiques ; echo "code=$?"                       [test
↪ d'erreur volontaire]
erreur : aucune valeur
usage : ./a4_statistiques <entier> [entier...]
code=1
sajith@ubuntu24-lab:~/tp2-c$ ./a4_statistiques 3 abc ; echo "code=$?"                 [test
↪ d'erreur volontaire]
erreur : valeur non numerique : abc
code=3
sajith@ubuntu24-lab:~/tp2-c$ ./a4_statistiques 3 99999999999 ; echo "code=$?"         [test
↪ d'erreur volontaire]
erreur : valeur hors des bornes d'un entier : 99999999999
code=4

Démonstration du piège de la division entière :
sajith@ubuntu24-lab:~/tp2-c$ ./a4_faux 12 7 30 -4                                     [mauvais
↪ calcul]
elements=4 somme=45 moyenne=11.00 min=-4 max=30
sajith@ubuntu24-lab:~/tp2-c$ ./a4_statistiques 12 7 30 -4                             [bon calcul]
elements=4 somme=45 moyenne=11.25 min=-4 max=30
sajith@ubuntu24-lab:~/tp2-c$ ./a4_faux 1 2                                            [mauvais
↪ calcul]
elements=2 somme=3 moyenne=1.00 min=1 max=2
sajith@ubuntu24-lab:~/tp2-c$ ./a4_statistiques 1 2                                    [bon calcul]
elements=2 somme=3 moyenne=1.50 min=1 max=2
sajith@ubuntu24-lab:~/tp2-c$ ./a4_faux 5 5 5                                          [division
↪ exacte : pas d'écart visible]
elements=3 somme=15 moyenne=5.00 min=5 max=5
```

Réponses aux questions

Question 4.1 — Que se passe-t-il concrètement si l'on écrit à l'indice 16 d'un tableau de 16 éléments ? Pourquoi le compilateur ne le signale-t-il pas toujours ?

Un tableau de 16 cases a les indices 0 à 15. L'indice 16 n'existe pas : c'est la case mémoire juste après le tableau. Je l'imagine comme une boîte à œufs de 16 places : poser un œuf en 17e position, c'est le poser sur la table d'à côté. Concrètement, l'écriture écrase une autre variable ou des données de la pile, et le programme peut donner des résultats faux, planter, ou être détourné. Le compilateur ne le voit pas toujours parce que le C ne vérifie pas les indices, et qu'ici l'indice dépend du nombre d'arguments tapés à l'exécution. Il ne peut avertir (-Warray-bounds) que si l'indice est une constante connue à la compilation. C'est pour ça que je compare n à MAX_VALEURS avant chaque écriture : avec 17 valeurs, le programme refuse.

Question 4.2 — Pourquoi a4_statistiques 5 5 5 5 donne-t-il 5.00 alors que a4_statistiques 5 5 5 donnerait 5 si l'on utilise une division entière ? Expliquez le mécanisme de conversion.

En C, si on divise deux entiers, le résultat est un entier : la partie après la virgule est jetée. La conversion en double ne se fait qu'après, au moment de ranger le résultat dans moyenne, donc trop tard. Ma version fausse le montre : 45 / 4 donne 11.00 au lieu de 11.25, et 3 / 2 donne 1.00 au lieu de 1.50. Avec 5 5 5 (15 / 3) ou 5 5 5 5 (20 / 4), la division tombe juste, il n'y a pas de reste, donc on ne voit aucune différence : on obtient 5, affiché 5.00. Pour corriger, j'écris (double)somme / n : la somme devient un double avant la division, n est converti aussi, et la division se fait avec les décimales.

Difficultés rencontrées

Aucune erreur de compilation. Le piège de la division entière n'apparaît pas sur tous les jeux de valeurs : avec 5 5 5, la version fausse donne le bon résultat car la division tombe juste. Il faut choisir un jeu dont la somme n'est pas divisible par le nombre d'éléments (12 7 30 -4, ou 1 2) pour mettre l'erreur en évidence.

Partie B — Mémoire et sécurité

B1 — Pointeurs et allocation dynamique

Objectif. Gérer la durée de vie de la mémoire : allouer à l'exécution, vérifier l'allocation, libérer, et le prouver par une analyse mémoire.

Fichiers. b1_statistiques_dyn.c

Choix documenté. Le tableau est alloué à la taille exacte du nombre de valeurs (argc - 1). Le produit n * sizeof(long) est contrôlé avant l'allocation pour éviter un débordement d'entier, le retour de malloc est testé, et la mémoire est libérée sur tous les chemins de sortie qui suivent l'allocation. Codes : 0 succès, 1 aucune valeur, 2 taille non représentable, 3 non numérique, 4 hors bornes, 5 échec d'allocation. Échec d'allocation : il a été provoqué en limitant l'espace d'adressage du processus avec prlimit --as. Une boucle a cherché la plus petite limite pour laquelle le programme démarre encore (en dessous, la bibliothèque C elle-même ne peut pas être chargée) : à 3 075 000 octets, malloc de 320 000 octets échoue et le programme rend le code 5 au lieu de planter.

Code source

b1_statistiques_dyn.c

```
sajith@ubuntu24-lab:~/tp2-c$ cat b1_statistiques_dyn.c
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
#include <errno.h>
#include <limits.h>

int main(int argc, char *argv[])
{
    if (argc < 2) {
        fprintf(stderr, "erreur : aucune valeur\nusage : %s <entier> [entier...]\n", argv[0]);
        return 1;
    }

    size_t n = (size_t)(argc - 1);

    if (n > SIZE_MAX / sizeof(long)) {
        fprintf(stderr, "erreur : trop de valeurs\n");
        return 2;
    }

    long *valeurs = malloc(n * sizeof(long));
    if (valeurs == NULL) {
        fprintf(stderr, "erreur : allocation de %zu octets impossible\n", n * sizeof(long));
        return 5;
    }

    for (size_t i = 0; i < n; i++) {
        char *fin;
        long v;

        errno = 0;
        v = strtol(argv[i + 1], &fin, 10);

        if (fin == argv[i + 1] || *fin != '\0') {
            fprintf(stderr, "erreur : valeur non numerique : %s\n", argv[i + 1]);
            free(valeurs);
            return 3;
        }
        if (errno == ERANGE || v < INT_MIN || v > INT_MAX) {
            fprintf(stderr, "erreur : valeur hors des bornes d'un entier : %s\n", argv[i + 1]);
            free(valeurs);
            return 4;
        }
```

```
        valeurs[i] = v;
    }

    long long somme = 0;
    long min = valeurs[0];
    long max = valeurs[0];

    for (size_t i = 0; i < n; i++) {
        somme = somme + valeurs[i];
        if (valeurs[i] < min) {
            min = valeurs[i];
        }
        if (valeurs[i] > max) {
            max = valeurs[i];
        }
    }

    double moyenne = (double)somme / (double)n;

    printf("elements=%zu somme=%lld moyenne=%.2f min=%ld max=%ld\n", n, somme, moyenne, min, max);

    free(valeurs);
    return 0;
}
```

Compilation

```
sajith@ubuntu24-lab:~/tp2-c$ gcc -Wall -Wextra -Werror -std=c17 -g -o b1_statistiques_dyn
↪ b1_statistiques_dyn.c
[aucune sortie = aucun avertissement]
sajith@ubuntu24-lab:~/tp2-c$ gcc -fsanitize=address,undefined -g -o b1_asan b1_statistiques_dyn.c
[aucune sortie]
```

Sorties obtenues

```
sajith@ubuntu24-lab:~/tp2-c$ ./b1_statistiques_dyn 12 7 30 -4 ; echo "code=$?"
elements=4 somme=45 moyenne=11.25 min=-4 max=30
code=0
sajith@ubuntu24-lab:~/tp2-c$ ./b1_statistiques_dyn $(seq 1 40000) ; echo "code=$?"          [40
↪ 000 valeurs]
elements=40000 somme=800020000 moyenne=20000.50 min=1 max=40000
code=0
sajith@ubuntu24-lab:~/tp2-c$ ./b1_statistiques_dyn ; echo "code=$?"
↪ [test d'erreur volontaire]
erreur : aucune valeur
usage : ./b1_statistiques_dyn <entier> [entier...]
code=1
sajith@ubuntu24-lab:~/tp2-c$ ./b1_statistiques_dyn 3 abc ; echo "code=$?"
↪ [test d'erreur volontaire]
erreur : valeur non numerique : abc
code=3

Analyse mémoire avec valgrind :
sajith@ubuntu24-lab:~/tp2-c$ valgrind --leak-check=full --track-fds=yes ./b1_statistiques_dyn 12
↪  7 30 -4
==1714== Memcheck, a memory error detector
==1714== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==1714== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==1714== Command: ./b1_statistiques_dyn 12 7 30 -4
==1714==
elements=4 somme=45 moyenne=11.25 min=-4 max=30
==1714==
==1714== FILE DESCRIPTORS: 3 open (3 std) at exit.
==1714==
==1714== HEAP SUMMARY:
==1714==     in use at exit: 0 bytes in 0 blocks
==1714==   total heap usage: 2 allocs, 2 frees, 1,056 bytes allocated
==1714==
==1714== All heap blocks were freed -- no leaks are possible
==1714==
==1714== For lists of detected and suppressed errors, rerun with: -s
==1714== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)

Analyse mémoire sur un chemin d'erreur (libération avant de quitter) :
sajith@ubuntu24-lab:~/tp2-c$ valgrind --leak-check=full ./b1_statistiques_dyn 3 abc 2>&1 | grep
↪ -E "ERROR SUMMARY|in use at exit|All heap"
==1715==     in use at exit: 0 bytes in 0 blocks
==1715== All heap blocks were freed -- no leaks are possible
==1715== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

```
Assainisseur d'adresses et de comportements indéfinis :
sajith@ubuntu24-lab:~/tp2-c$ ./b1_asan 12 7 30 -4 ; echo "code=$?"
elements=4 somme=45 moyenne=11.25 min=-4 max=30
code=0
[aucun rapport d'erreur de l'assainisseur]

Échec d'allocation provoqué (limite de l'espace d'adressage avec prlimit) :
sajith@ubuntu24-lab:~/tp2-c$ prlimit --as=3075000 ./b1_statistiques_dyn $(seq 1 40000)
erreur : allocation de 320000 octets impossible
code=5
```

Réponses aux questions

Question 5.1 — Quelle valeur rend malloc en cas d'échec, et pourquoi le programme doit-il la tester avant toute écriture ?

malloc rend NULL quand il n'arrive pas à allouer. NULL ne pointe vers aucune mémoire utilisable : écrire dedans, c'est comme vouloir déposer ses affaires dans un garde-meuble qui n'existe pas. Le programme plante, ou pire, a un comportement imprévisible. Je teste donc valeurs == NULL juste après l'appel : dans ce cas, j'affiche un message et je quitte avec le code 5. Je l'ai vérifié en limitant la mémoire avec prlimit : le programme rend bien 5 au lieu de planter.

Question 5.2 — Un programme utilise la mémoire après l'avoir libérée et « fonctionne » pendant les tests. Expliquez pourquoi ce n'est pas une preuve de correction, et quel outil permet de le détecter.

Après free, la mémoire n'est pas effacée, elle est juste rendue. C'est comme rendre les clés d'un garde-meuble : mes affaires y sont encore un moment, mais le prochain locataire peut tout remplacer. Tant que personne n'a réutilisé la zone, le programme lit encore les anciennes valeurs et semble marcher. Mais dès qu'une autre allocation la récupère (autre charge, autre machine, production), les données deviennent fausses. Un test qui passe ne prouve donc rien. On le détecte avec valgrind, qui signale des « Invalid read » ou « Invalid write », ou avec l'assainisseur d'adresses (-fsanitize=address), qui arrête le programme sur un « heap-use-after-free ».

Difficultés rencontrées

Le cas d'échec d'allocation est difficile à déclencher. Une limite posée avec ulimit -v s'applique aussi au shell, qui ne peut alors plus fonctionner ; et une limite trop basse empêche le programme de démarrer (la bibliothèque C ne peut pas être chargée). La solution retenue : prlimit, qui n'applique la limite qu'au programme lancé, et une boucle qui cherche une limite intermédiaire où le programme démarre mais où malloc échoue. Le contrôle du débordement de n * sizeof(long) n'a pas pu être déclenché : le nombre d'arguments est borné par le système bien avant d'atteindre SIZE_MAX / sizeof(long).

B2 — Débordement de tampon : du programme vulnérable au programme sûr

Objectif. Reproduire la faille la plus ancienne, la mettre en évidence par outil, puis la corriger en refusant toute entrée trop longue.

Fichiers. b2_vulnerable.c, b2_sur.c

Choix documenté. Pourquoi la compilation silencieuse rend cette faille dangereuse : strcpy est une fonction parfaitement valide pour le compilateur, qui ne connaît pas la longueur de argv[1] puisqu'elle n'est fixée qu'à l'exécution. Le programme compile donc sans aucun avertissement et fonctionne normalement avec des entrées courtes, si bien que les tests habituels ne révèlent rien. Le défaut n'apparaît qu'avec une entrée longue fournie de l'extérieur, c'est-à-dire précisément entre les mains d'un attaquant, en production.

Code source

b2_vulnerable.c (programme fourni, volontairement défectueux)

```
sajith@ubuntu24-lab:~/tp2-c$ cat b2_vulnerable.c
#include <stdio.h>
#include <string.h>

int main(int argc, char *argv[])
{
    char tampon[16];
    if (argc != 2) return 2;

    strcpy(tampon, argv[1]);
    printf("recu : %s\n", tampon);
    return 0;
}
```

b2_sur.c (version corrigée)

```
sajith@ubuntu24-lab:~/tp2-c$ cat b2_sur.c
#include <stdio.h>
#include <string.h>

#define CAPACITE 16

int main(int argc, char *argv[])
{
    char tampon[CAPACITE];

    if (argc != 2) {
        fprintf(stderr, "usage : %s <texte>\n", argv[0]);
        return 2;
    }

    size_t longueur = strlen(argv[1]);

    if (longueur >= CAPACITE) {
        fprintf(stderr, "entree trop longue (%zu octets, maximum %d) - refusee\n", longueur,
↪ CAPACITE - 1);
        return 3;
    }

    snprintf(tampon, sizeof(tampon), "%s", argv[1]);
    printf("recu : %s\n", tampon);
    return 0;
}
```

Compilation

```
Version vulnérable (options du sujet, sans -Werror) :
sajith@ubuntu24-lab:~/tp2-c$ gcc -Wall -Wextra -std=c17 -g -o b2_vulnerable b2_vulnerable.c
[aucune sortie : aucun avertissement, code de retour 0]
sajith@ubuntu24-lab:~/tp2-c$ gcc -fsanitize=address -g -o b2_vulnerable_asan b2_vulnerable.c

Version corrigée :
sajith@ubuntu24-lab:~/tp2-c$ gcc -Wall -Wextra -Werror -std=c17 -g -o b2_sur b2_sur.c
[aucune sortie = aucun avertissement]
sajith@ubuntu24-lab:~/tp2-c$ gcc -fsanitize=address -g -o b2_sur_asan b2_sur.c
```

Sorties obtenues

```
Étape 1 — Constater (version vulnérable)
sajith@ubuntu24-lab:~/tp2-c$ ./b2_vulnerable court ; echo "code=$?"
recu : court
code=0
sajith@ubuntu24-lab:~/tp2-c$ ./b2_vulnerable AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA ; echo
↪ "code=$?"     [40 caractères]
recu : AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
*** stack smashing detected ***: terminated
Aborted (core dumped)
code=134

Étape 2 — Prouver (assainisseur d'adresses sur la version vulnérable)
sajith@ubuntu24-lab:~/tp2-c$ ./b2_vulnerable_asan AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
=================================================================
==1855==ERROR: AddressSanitizer: stack-buffer-overflow on address 0x744f28100030 at pc
↪ 0x744f2a6a7923 bp 0x7ffeff18fd90 sp 0x7ffeff18f538
WRITE of size 41 at 0x744f28100030 thread T0
    #0 0x744f2a6a7922 in strcpy ../../../../src/libsanitizer/asan/asan_interceptors.cpp:563
    #1 0x6136a2b37329 in main /home/sajith/tp2-c/b2_vulnerable.c:9
    #2 0x744f2a22a1c9 in __libc_start_call_main ../sysdeps/nptl/libc_start_call_main.h:58
    #3 0x744f2a22a28a in __libc_start_main_impl ../csu/libc-start.c:360
    #4 0x6136a2b37184 in _start (/home/sajith/tp2-c/b2_vulnerable_asan+0x1184)

Address 0x744f28100030 is located in stack of thread T0 at offset 48 in frame
    #0 0x6136a2b37258 in main /home/sajith/tp2-c/b2_vulnerable.c:5

  This frame has 1 object(s):
    [32, 48) 'tampon' (line 6) <== Memory access at offset 48 overflows this variable
SUMMARY: AddressSanitizer: stack-buffer-overflow
↪ ../../../../src/libsanitizer/asan/asan_interceptors.cpp:563 in strcpy
[...rapport tronqué : carte de la mémoire fantôme]

Étape 3 — Corriger (version sûre)
sajith@ubuntu24-lab:~/tp2-c$ ./b2_sur court ; echo "code=$?"
recu : court
code=0
sajith@ubuntu24-lab:~/tp2-c$ ./b2_sur 123456789012345 ; echo "code=$?"                    [15
↪ caractères : limite acceptée]
recu : 123456789012345
code=0
sajith@ubuntu24-lab:~/tp2-c$ ./b2_sur 1234567890123456 ; echo "code=$?"                   [16
↪ caractères : premier refus]
entree trop longue (16 octets, maximum 15) - refusee
code=3
sajith@ubuntu24-lab:~/tp2-c$ ./b2_sur AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA ; echo "code=$?"
```

```
entree trop longue (40 octets, maximum 15) - refusee
code=3
sajith@ubuntu24-lab:~/tp2-c$ ./b2_sur AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA > sortie.txt ;
↪ echo "code=$?"
entree trop longue (40 octets, maximum 15) - refusee
code=3
sajith@ubuntu24-lab:~/tp2-c$ wc -c sortie.txt                                            [rien
↪ sur la sortie standard]
0 sortie.txt

Étape 4 — Comparer (assainisseur sur la version sûre, 10 000 octets)
sajith@ubuntu24-lab:~/tp2-c$ ./b2_sur_asan "$(head -c 10000 /dev/zero | tr '\0' A)" ; echo
↪ "code=$?"
entree trop longue (10000 octets, maximum 15) - refusee
code=3
[aucun rapport de l'assainisseur]
```

Réponses aux questions

Question 6.1 — Pourquoi le compilateur n'avertit-il pas sur la ligne strcpy(tampon, argv[1]) ? Que faudrait-il pour qu'il le fasse, et quelles en sont les limites ?

Le compilateur connaît la taille du tampon (16 octets), mais pas celle de argv[1], qui dépend de ce que l'utilisateur tape à l'exécution. Pour lui, strcpy est un appel tout à fait valide, donc il ne dit rien. C'est comme connaître la taille du verre sans savoir combien on va verser. Pour qu'il avertisse, il faudrait que la longueur de la source soit connue à la compilation, ou activer des protections en plus

: _FORTIFY_SOURCE avec -O2, qui remplace strcpy par une version vérifiée, ou l'analyseur -fanalyzer. Mais ces outils ne voient que ce qui est prouvable à la compilation, et _FORTIFY_SOURCE se contente d'arrêter le programme quand le débordement arrive. La seule vraie protection, c'est de comparer la longueur à la capacité dans mon code.

Question 6.2 — Expliquez en vos propres mots la différence de comportement entre strncpy et snprintf lorsqu'une chaîne source est plus longue que la destination. Laquelle utilisez-vous désormais, et pourquoi ?

Les deux s'arrêtent à la taille de la destination, mais strncpy a un piège : si la source est trop longue, elle copie exactement n caractères et n'ajoute pas le zéro de fin. La chaîne n'a donc pas de fin, et la fonction suivante qui la lit (printf, strlen) continue au-delà du tampon. snprintf, lui, copie au plus taille - 1 caractères et met toujours le zéro final : le résultat est toujours une vraie chaîne. En plus, il rend la longueur qu'il aurait voulu écrire, ce qui permet de voir qu'il a coupé. J'utilise maintenant snprintf, et je vérifie la longueur avant pour refuser l'entrée au lieu de la couper.

Question 6.3 — En quoi le fait de refuser une entrée trop longue est-il préférable à sa troncature, du point de vue d'un auditeur de sécurité ?

Une entrée trop longue, ce n'est pas normal : soit c'est une erreur, soit c'est une tentative d'attaque. La refuser, c'est sonner l'alarme : il y a un message sur la sortie d'erreur et un code de sortie à part (3), donc on peut le repérer, le journaliser et enquêter. Couper en silence, au contraire, cache la tentative, et le programme continue avec une donnée que personne n'a vraiment tapée. Un nom de fichier coupé, par exemple, peut désigner un autre fichier. Pour un auditeur, un refus se vérifie ; une troncature, c'est un comportement caché.

Difficultés rencontrées

La version vulnérable n'a pas planté au moment de la copie : elle a d'abord affiché « recu : AAAA... » comme si tout allait bien, et ne s'est arrêtée qu'au retour de main, lorsque la protection de pile d'Ubuntu a constaté la corruption (stack smashing detected, code 134 = 128 + SIGABRT). Cette protection ne fait que constater les dégâts après coup : c'est l'assainisseur d'adresses qui a désigné l'écriture fautive (strcpy, ligne 9). Pour la version sûre, j'ai testé la limite exacte : 15 caractères acceptés, 16 refusés.

Partie C — Fichiers et appels système

C1 — Fichiers tamponnés et comptage

Objectif. Lire un fichier par blocs bornés sans le charger entièrement, et distinguer fin de fichier et erreur de lecture.

Fichiers. c1_compteur.c

Choix documenté. Le fichier est lu avec fgets dans un tampon fixe de 64 octets : il n'est jamais chargé entièrement en mémoire, et une ligne plus longue que le tampon est lue en plusieurs morceaux (lignes comptées sur les \n, état « dans un mot » conservé d'un morceau à l'autre). Après la boucle, ferror distingue l'erreur de lecture de la fin de fichier normale ; fclose est vérifié. Codes : 0 succès, 1 usage, 2 ouverture impossible (fichier absent), 3 pas un fichier ordinaire (vérifié avec fstat et S_ISREG), 4 erreur de lecture, 5 erreur de fermeture. L'erreur de lecture est démontrée avec /proc/self/mem, un fichier ordinaire qui s'ouvre mais dont la lecture échoue (Input/output error).

Code source

c1_compteur.c

```
sajith@ubuntu24-lab:~/tp2-c$ cat c1_compteur.c
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <string.h>
#include <errno.h>
#include <ctype.h>
#include <sys/stat.h>

#define TAILLE_TAMPON 64

int main(int argc, char *argv[])
{
    if (argc != 2) {
        fprintf(stderr, "usage : %s <fichier>\n", argv[0]);
        return 1;
    }

    FILE *f = fopen(argv[1], "r");
    if (f == NULL) {
        fprintf(stderr, "erreur : ouverture de %s impossible : %s\n", argv[1], strerror(errno));
        return 2;
    }

    struct stat infos;
    if (fstat(fileno(f), &infos) != 0 || !S_ISREG(infos.st_mode)) {
        fprintf(stderr, "erreur : %s n'est pas un fichier ordinaire\n", argv[1]);
        fclose(f);
        return 3;
    }

    char tampon[TAILLE_TAMPON];
    long lignes = 0;
    long mots = 0;
    long octets = 0;
    int dans_un_mot = 0;

    while (fgets(tampon, sizeof(tampon), f) != NULL) {
        size_t n = strlen(tampon);
        octets = octets + (long)n;

        for (size_t i = 0; i < n; i++) {
            unsigned char c = (unsigned char)tampon[i];
            if (c == '\n') {
                lignes++;
            }
```

```
            if (isspace(c)) {
                dans_un_mot = 0;
            } else if (!dans_un_mot) {
                dans_un_mot = 1;
                mots++;
            }
        }
    }

    if (ferror(f)) {
        fprintf(stderr, "erreur : lecture de %s interrompue : %s\n", argv[1], strerror(errno));
        fclose(f);
        return 4;
    }

    if (fclose(f) != 0) {
        fprintf(stderr, "erreur : fermeture de %s : %s\n", argv[1], strerror(errno));
        return 5;
    }

    printf("lignes=%ld mots=%ld octets=%ld %s\n", lignes, mots, octets, argv[1]);
    return 0;
}
```

Compilation

```
sajith@ubuntu24-lab:~/tp2-c$ gcc -Wall -Wextra -Werror -std=c17 -g -o c1_compteur c1_compteur.c
[aucune sortie = aucun avertissement]
```

Sorties obtenues

```
Comparaison avec wc sur quatre fichiers :
sajith@ubuntu24-lab:~/tp2-c$ ./c1_compteur /etc/os-release ; echo "code=$?" ; wc /etc/os-release
↪       [fichier normal]
lignes=13 mots=18 octets=400 /etc/os-release
code=0
 13  18 400 /etc/os-release
sajith@ubuntu24-lab:~/tp2-c$ ./c1_compteur vide.txt ; echo "code=$?" ; wc vide.txt
↪                     [fichier vide, créé avec touch]
lignes=0 mots=0 octets=0 vide.txt
code=0
0 0 0 vide.txt
sajith@ubuntu24-lab:~/tp2-c$ ./c1_compteur sans_fin.txt ; echo "code=$?" ; wc sans_fin.txt
↪             [printf 'un deux\ntrois quatre' : pas de \n final]
lignes=1 mots=4 octets=20 sans_fin.txt
code=0
 1  4 20 sans_fin.txt
sajith@ubuntu24-lab:~/tp2-c$ ./c1_compteur ligne_longue.txt ; echo "code=$?" ; wc
↪ ligne_longue.txt    [ligne de 300 caractères > tampon de 64]
lignes=1 mots=1 octets=301 ligne_longue.txt
code=0
  1   1 301 ligne_longue.txt

Cas d'erreur :
sajith@ubuntu24-lab:~/tp2-c$ ./c1_compteur /etc ; echo "code=$?"
↪                                       [test d'erreur volontaire : répertoire]
erreur : /etc n'est pas un fichier ordinaire
code=3
sajith@ubuntu24-lab:~/tp2-c$ ./c1_compteur /fichier/inexistant ; echo "code=$?"
↪                          [test d'erreur volontaire : fichier absent]
erreur : ouverture de /fichier/inexistant impossible : No such file or directory
code=2
sajith@ubuntu24-lab:~/tp2-c$ ./c1_compteur /proc/self/mem ; echo "code=$?"
↪                               [test d'erreur volontaire : erreur de lecture]
erreur : lecture de /proc/self/mem interrompue : Input/output error
code=4
sajith@ubuntu24-lab:~/tp2-c$ ./c1_compteur ; echo "code=$?"
↪                                           [test d'erreur volontaire]
usage : ./c1_compteur <fichier>
code=1
```

Réponses aux questions

Question 7.1 — Pourquoi la fonction gets a-t-elle été supprimée du langage ? Quelle fonction la remplace et quelle information supplémentaire faut-il lui fournir ?

gets lisait une ligne dans un tampon sans connaître sa taille : elle copiait tout jusqu'au retour à la ligne. Une ligne trop longue débordait donc forcément, exactement comme le strcpy de B2, et il n'y avait aucun moyen de l'utiliser sans risque. Elle a été déclarée obsolète en C99, puis supprimée en C11. On la remplace par fgets, à qui on donne en plus la taille du tampon (et le flux à lire) : fgets(tampon, sizeof(tampon), f) lit au plus taille - 1 caractères et termine toujours la chaîne par un zéro.

Question 7.2 — Sur un fichier sans retour à la ligne final, votre comptage diffère-t-il de celui de wc ? Expliquez l'écart observé, ou justifiez l'absence d'écart.

Non, il n'y a pas d'écart : pour sans_fin.txt, mon programme et wc donnent tous les deux 1 ligne, 4 mots et 20 octets. À l'écran, le fichier a pourtant 2 lignes (« un deux » et « trois quatre »), mais il ne contient qu'un seul retour à la ligne. wc ne compte pas les lignes qu'on voit, il compte les caractères \n, et mon programme fait pareil. Si j'avais compté les appels réussis à fgets, j'aurais trouvé 2 pour ce fichier, et 5 pour ligne_longue.txt, parce que fgets lit une ligne de 300 caractères en plusieurs morceaux avec mon tampon de 64.

Difficultés rencontrées

Deux points délicats. D'abord, fopen réussit sur un répertoire sous Linux : c'est la lecture qui échoue ensuite. Pour refuser /etc proprement, avec son propre code, le type est vérifié juste après l'ouverture avec fstat et S_ISREG. Ensuite, fstat et fileno ne font pas partie du C standard : avec -std=c17, il faut la ligne #define _POSIX_C_SOURCE 200809L avant les inclusions. Enfin, déclencher une vraie erreur de lecture n'est pas évident ; /proc/self/mem le permet.

C2 — Appels système bruts : reprendre le programme du CM 1

Objectif. Copier un fichier avec open/read/write en gérant les écritures partielles et les interruptions, puis mesurer l'effet de la taille de bloc.

Fichiers. c2_copie.c

Choix documenté. Robustesse ajoutée au programme du CM 1 : chaque open, read et write interrompu (retour -1 avec errno = EINTR) est rejoué ; la boucle d'écriture continue à partir de tampon + ecrits_total jusqu'à ce que tout le bloc lu soit écrit ; les deux descripteurs sont ouverts avec O_CLOEXEC ; les fermetures sont vérifiées ; le tampon est alloué sur le tas (malloc) et borné à 1 Mio pour ne pas risquer de dépasser la pile. Options : --verbeux (compteurs sur la sortie d'erreur) et --bloc N (1 à 1 048 576). Pour provoquer les interruptions, le programme installe un gestionnaire de SIGUSR1 sans SA_RESTART ; le signal est envoyé avec kill pendant que le programme est bloqué sur un tube nommé (mkfifo), en lecture puis en écriture. Écarts entre le mode verbeux et strace : strace compte 214 read contre 213 annoncés, car le chargeur dynamique lit l'en-tête de libc.so.6 avant main ; il compte 4 openat et 4 close, dont 2 du chargeur (ld.so.cache et libc.so.6). Les 212 write sont identiques. Les 213 lectures correspondent aux 212 blocs de données (866 987 / 4096 = 211,7) plus la lecture finale qui rend 0 (fin de fichier).

Code source

c2_copie.c

```
sajith@ubuntu24-lab:~/tp2-c$ cat c2_copie.c
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <errno.h>
#include <fcntl.h>
#include <unistd.h>
#include <signal.h>

#define BLOC_DEFAUT 4096
#define BLOC_MAX 1048576

static volatile sig_atomic_t signaux_recus = 0;

static void gestionnaire(int sig)
{
    (void)sig;
    signaux_recus++;
}

int main(int argc, char *argv[])
{
    int verbeux = 0;
    long bloc = BLOC_DEFAUT;
    int i = 1;

    while (i < argc && strncmp(argv[i], "--", 2) == 0) {
        if (strcmp(argv[i], "--verbeux") == 0) {
            verbeux = 1;
            i++;
        } else if (strcmp(argv[i], "--bloc") == 0 && i + 1 < argc) {
            char *fin;
            errno = 0;
            bloc = strtol(argv[i + 1], &fin, 10);
            if (fin == argv[i + 1] || *fin != '\0' || errno == ERANGE || bloc < 1 || bloc >
↪ BLOC_MAX) {
                fprintf(stderr, "erreur : taille de bloc invalide : %s (1 a %d)\n", argv[i + 1],
↪  BLOC_MAX);
                return 1;
            }
            i += 2;
        } else {
            fprintf(stderr, "erreur : option inconnue : %s\n", argv[i]);
            return 1;
```

```
        }
    }

    if (argc - i != 2) {
        fprintf(stderr, "usage : %s [--verbeux] [--bloc N] <source> <destination>\n", argv[0]);
        return 1;
    }

    const char *source = argv[i];
    const char *destination = argv[i + 1];

    struct sigaction sa;
    memset(&sa, 0, sizeof(sa));
    sa.sa_handler = gestionnaire;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;
    sigaction(SIGUSR1, &sa, NULL);

    char *tampon = malloc((size_t)bloc);
    if (tampon == NULL) {
        fprintf(stderr, "erreur : allocation du tampon impossible\n");
        return 2;
    }

    int fd_src;
    do {
        fd_src = open(source, O_RDONLY | O_CLOEXEC);
    } while (fd_src == -1 && errno == EINTR);
    if (fd_src == -1) {
        fprintf(stderr, "erreur : ouverture de %s : %s\n", source, strerror(errno));
        free(tampon);
        return 3;
    }

    int fd_dst;
    do {
        fd_dst = open(destination, O_WRONLY | O_CREAT | O_TRUNC | O_CLOEXEC, 0644);
    } while (fd_dst == -1 && errno == EINTR);
    if (fd_dst == -1) {
        fprintf(stderr, "erreur : ouverture de %s : %s\n", destination, strerror(errno));
        close(fd_src);
        free(tampon);
        return 4;
    }
```

```
    long lectures = 0;
    long ecritures = 0;
    long partielles = 0;
    long interruptions = 0;
    long long total = 0;
    int code = 0;

    for (;;) {
        ssize_t lus = read(fd_src, tampon, (size_t)bloc);
        if (lus == -1) {
            if (errno == EINTR) {
                interruptions++;
                continue;
            }
            fprintf(stderr, "erreur : lecture de %s : %s\n", source, strerror(errno));
            code = 5;
            break;
        }
        lectures++;
        if (lus == 0) {
            break;
        }

        ssize_t ecrits_total = 0;
        while (ecrits_total < lus) {
            ssize_t ecrits = write(fd_dst, tampon + ecrits_total, (size_t)(lus - ecrits_total));
            if (ecrits == -1) {
                if (errno == EINTR) {
                    interruptions++;
                    continue;
                }
                fprintf(stderr, "erreur : ecriture dans %s : %s\n", destination, strerror(errno));
                code = 6;
                break;
            }
            ecritures++;
            if (ecrits < lus - ecrits_total) {
                partielles++;
                if (verbeux) {
                    fprintf(stderr, "ecriture partielle : %zd octets sur %zd, reprise a l'octet
↪ %zd\n",
                            ecrits, lus - ecrits_total, ecrits_total + ecrits);
                }
            }
            ecrits_total += ecrits;
        }
        if (code != 0) {
            break;
        }
        total += lus;
    }

    if (close(fd_src) == -1 && code == 0) {
        fprintf(stderr, "erreur : fermeture de %s : %s\n", source, strerror(errno));
        code = 7;
    }
    if (close(fd_dst) == -1 && code == 0) {
        fprintf(stderr, "erreur : fermeture de %s : %s\n", destination, strerror(errno));
        code = 7;
    }
    free(tampon);

    if (verbeux) {
        fprintf(stderr, "bloc=%ld lectures=%ld ecritures=%ld partielles=%ld interruptions=%ld
↪ octets=%lld\n",
                bloc, lectures, ecritures, partielles, interruptions, total);
    }
    return code;
}
```

Compilation

```
sajith@ubuntu24-lab:~/tp2-c$ gcc -Wall -Wextra -Werror -std=c17 -g -o c2_copie c2_copie.c
[aucune sortie = aucun avertissement]
```

Sorties obtenues

```
sajith@ubuntu24-lab:~/tp2-c$ ./c2_copie --verbeux /var/log/dpkg.log /tmp/copie1 ; echo "code=$?"
bloc=4096 lectures=213 ecritures=212 partielles=0 interruptions=0 octets=866987
code=0
sajith@ubuntu24-lab:~/tp2-c$ cmp /var/log/dpkg.log /tmp/copie1 && echo "contenu identique"
contenu identique

Appels réellement exécutés (strace) :
sajith@ubuntu24-lab:~/tp2-c$ strace -c -e trace=read,write,openat,close ./c2_copie
↪ /var/log/dpkg.log /tmp/copie2
% time     seconds  usecs/call     calls    errors syscall
------ ----------- ----------- --------- --------- ----------------
 53,95    0,007821          36       214           read
 45,08    0,006536          30       212           write
  0,57    0,000082          20         4           openat
  0,41    0,000059          14         4           close
------ ----------- ----------- --------- --------- ----------------
100,00    0,014498          33       434           total

Preuve du drapeau O_CLOEXEC :
sajith@ubuntu24-lab:~/tp2-c$ strace -e trace=openat ./c2_copie /var/log/dpkg.log /tmp/copie2
↪ 2>&1 | grep -E "dpkg|copie2"
openat(AT_FDCWD, "/var/log/dpkg.log", O_RDONLY|O_CLOEXEC) = 3
openat(AT_FDCWD, "/tmp/copie2", O_WRONLY|O_CREAT|O_TRUNC|O_CLOEXEC, 0644) = 4

Cas d'erreur :
sajith@ubuntu24-lab:~/tp2-c$ ./c2_copie /fichier/inexistant /tmp/x ; echo "code=$?"
↪                  [test d'erreur volontaire]
erreur : ouverture de /fichier/inexistant : No such file or directory
code=3
sajith@ubuntu24-lab:~/tp2-c$ ./c2_copie --bloc 0 a b ; echo "code=$?"
↪                                [test d'erreur volontaire]
erreur : taille de bloc invalide : 0 (1 a 1048576)
code=1

Interruption provoquée pendant une lecture (tube nommé, signal SIGUSR1) :
sajith@ubuntu24-lab:~/tp2-c$ mkfifo /tmp/tube
sajith@ubuntu24-lab:~/tp2-c$ ./c2_copie --verbeux /tmp/tube /tmp/recu_tube &            [le
↪ programme attend des données dans read]
[1] 1936
sajith@ubuntu24-lab:~/tp2-c$ exec 3>/tmp/tube ; sleep 1
sajith@ubuntu24-lab:~/tp2-c$ kill -USR1 1936                                            [read
↪ rend -1 avec errno = EINTR : il est rejoué]
sajith@ubuntu24-lab:~/tp2-c$ echo "donnees envoyees apres interruption" >&3 ; exec 3>&-
bloc=4096 lectures=2 ecritures=1 partielles=0 interruptions=1 octets=36
code=0
```

```
sajith@ubuntu24-lab:~/tp2-c$ cat /tmp/recu_tube
donnees envoyees apres interruption

Écriture partielle provoquée (bloc de 1 Mio vers un tube de 64 Kio non lu) :
sajith@ubuntu24-lab:~/tp2-c$ head -c 3M /dev/urandom > /tmp/gros.bin
sajith@ubuntu24-lab:~/tp2-c$ mkfifo /tmp/tube2
sajith@ubuntu24-lab:~/tp2-c$ ( exec 4</tmp/tube2; sleep 2; cat <&4 > /tmp/recu2 ) &     [lecteur
↪  qui attend 2 s avant de vider le tube]
sajith@ubuntu24-lab:~/tp2-c$ ./c2_copie --verbeux --bloc 1048576 /tmp/gros.bin /tmp/tube2 &
[2] 1944
sajith@ubuntu24-lab:~/tp2-c$ sleep 1 ; kill -USR1 1944                                    [le
↪ programme est bloqué dans write]
ecriture partielle : 65536 octets sur 1048576, reprise a l'octet 65536
bloc=1048576 lectures=4 ecritures=4 partielles=1 interruptions=0 octets=3145728
code=0
sajith@ubuntu24-lab:~/tp2-c$ cmp /tmp/gros.bin /tmp/recu2 && echo "contenu identique"
contenu identique

Mesures sur un fichier de 50 Mio (copie vérifiée par cmp à chaque fois) :
sajith@ubuntu24-lab:~/tp2-c$ head -c 50M /dev/urandom > /tmp/source50
sajith@ubuntu24-lab:~/tp2-c$ TIMEFORMAT='temps reel=%R s  (utilisateur=%U s, systeme=%S s)'
sajith@ubuntu24-lab:~/tp2-c$ for b in 65536 4096 64 1; do time ./c2_copie --verbeux --bloc $b
↪ /tmp/source50 /tmp/copie50; cmp /tmp/source50 /tmp/copie50 && echo "copie identique"; done
--- bloc=65536
bloc=65536 lectures=801 ecritures=800 partielles=0 interruptions=0 octets=52428800
temps reel=0,376 s  (utilisateur=0,000 s, systeme=0,376 s)
copie identique
--- bloc=4096
bloc=4096 lectures=12801 ecritures=12800 partielles=0 interruptions=0 octets=52428800
temps reel=0,056 s  (utilisateur=0,000 s, systeme=0,056 s)
copie identique
--- bloc=64
bloc=64 lectures=819201 ecritures=819200 partielles=0 interruptions=0 octets=52428800
temps reel=0,732 s  (utilisateur=0,156 s, systeme=0,576 s)
copie identique
--- bloc=1
bloc=1 lectures=52428801 ecritures=52428800 partielles=0 interruptions=0 octets=52428800
temps reel=43,423 s  (utilisateur=8,829 s, systeme=34,570 s)
copie identique

Nouvelle mesure des deux grands blocs (fichier destination déjà existant) :
sajith@ubuntu24-lab:~/tp2-c$ for essai in 1 2 3; do for b in 65536 4096; do time ./c2_copie
↪ --bloc $b /tmp/source50 /tmp/copie50; done; done
--- essai 1, bloc=65536
temps reel=0,068 s  (utilisateur=0,001 s, systeme=0,066 s)
```

```
--- essai 1, bloc=4096
temps reel=0,052 s  (utilisateur=0,003 s, systeme=0,048 s)
--- essai 2, bloc=65536
temps reel=0,041 s  (utilisateur=0,000 s, systeme=0,041 s)
--- essai 2, bloc=4096
temps reel=0,047 s  (utilisateur=0,009 s, systeme=0,039 s)
--- essai 3, bloc=65536
temps reel=0,047 s  (utilisateur=0,000 s, systeme=0,047 s)
--- essai 3, bloc=4096
temps reel=0,054 s  (utilisateur=0,004 s, systeme=0,050 s)
```

Tableau des mesures — copie de 50 Mio (52 428 800 octets)

Bloc (octets)	Lectures	Écritures	Appels read + write	Temps réel (1re mesure)	Temps réel (3 nouvelles mesures)
1	52 428 801	52 428 800	104 857 601	43,423 s (dont 34,570 s système)	—
64	819 201	819 200	1 638 401	0,732 s	—
4096	12 801	12 800	25 601	0,056 s	0,052 / 0,047 / 0,054 s
65536	801	800	1 601	0,376 s (1re copie : biais)	0,068 / 0,041 / 0,047 s

Conclusion des mesures

Le nombre d'appels est divisé par la taille du bloc : 105 millions d'appels avec 1 octet, 1 601 avec 64 Kio. Le temps suit tant que le coût des appels domine : de 1 à 64 octets, 64 fois moins d'appels donnent un temps 59 fois plus court (43,4 s contre 0,73 s), et presque tout ce temps est passé dans le noyau (34,6 s de temps système). Chaque appel coûte environ 0,4 µs (43,4 s / 105 millions), quelle que soit la quantité transférée : c'est le prix fixe du passage en mode noyau. À partir de 4096 octets, le nombre d'appels devient négligeable et le temps se stabilise autour de 0,05 s : il est alors dominé par la copie des 50 Mio elle-même, et passer à 64 Kio ne change presque plus rien (écarts de l'ordre du bruit de mesure). La première mesure à 64 Kio (0,376 s) est un biais : c'était la première copie, qui a dû créer le fichier destination et lui réserver 50 Mio ; les trois mesures suivantes, sur un fichier existant, donnent 0,041 à 0,068 s. Taille de bloc retenue : 64 Kio (65 536 octets). Elle réduit le nombre d'appels au minimum pour un coût mémoire faible ; 4096 octets, la taille d'une page mémoire, est un minimum acceptable. Le tampon est alloué sur le tas : un tampon de cette taille sur la pile, répété dans des appels imbriqués, risquerait de dépasser la limite de pile du processus.

Réponses aux questions

Question 8.1 — Pourquoi un appel interrompu par un signal doit-il être rejoué, et non traité comme une erreur ? Quelle valeur particulière faut-il tester ?

Quand un signal arrive pendant qu'un appel système attend (un read qui attend des données, par exemple), le noyau interrompt l'appel pour exécuter le gestionnaire. Si rien n'a encore été transféré, l'appel rend -1 avec errno égal à EINTR. Ce n'est pas une vraie erreur : le fichier va bien, l'opération n'a simplement pas eu lieu, il faut la refaire. La traiter comme une erreur ferait échouer une copie qui marchait, juste à cause d'un signal extérieur. Il faut donc tester errno == EINTR. Ma démonstration avec le tube et kill -USR1 le montre : interruptions=1, puis la lecture est relancée et les données arrivent normalement.

Question 8.2 — D'après vos mesures, de combien le temps varie-t-il entre la plus petite et la plus grande taille de bloc ? Reliez cet écart au coût du franchissement de la frontière du noyau décrit au chapitre 2.3 du CM 1.

Avec un bloc de 1 octet, la copie de 50 Mio prend 43,4 s. Avec 64 Kio, environ 0,05 s (entre 0,041 et 0,068 s sur trois essais). C'est à peu près 900 fois plus rapide, alors que ce sont exactement les mêmes données. Ce qui change, c'est le nombre d'appels système : 105 millions contre 1 601. Chaque read ou write fait passer la frontière entre mon programme et le noyau, et ce passage a un coût fixe, environ 0,4 µs ici, que l'on copie 1 octet ou 64 Kio. Avec des blocs de 1 octet, ce coût est payé 105 millions de fois : on le voit bien avec 34,6 s de temps système sur 43,4 s. Avec de gros blocs, il devient négligeable.

Question 8.3 — Que signifie le drapeau O_CLOEXEC et quel risque précis évite-t-il ?

O_CLOEXEC veut dire « close on exec » : le fichier est fermé automatiquement si le programme est remplacé par un autre avec exec. Sans ce drapeau, le nouveau programme hérite des fichiers déjà ouverts. Le risque, c'est la fuite de descripteur : un programme lancé ensuite peut lire ou écrire dans un fichier qu'il n'aurait jamais eu le droit d'ouvrir, par exemple un fichier confidentiel ouvert par un programme root. Comme le drapeau est donné directement à l'ouverture, il n'y a même pas de moment où le fichier est ouvert sans protection. strace montre qu'il est bien appliqué sur mes deux ouvertures.

Difficultés rencontrées

Deux difficultés de démonstration. Sur un fichier ordinaire, read et write ne sont pas interrompus par un signal (le disque répond trop vite) : il a fallu un tube nommé, sur lequel le programme reste bloqué, pour provoquer EINTR, puis un bloc plus grand que la capacité du tube (64 Kio) pour obtenir une écriture partielle. Une écriture interrompue après un premier transfert ne rend pas EINTR mais le nombre d'octets déjà écrits : c'est ce qui produit l'écriture partielle (interruptions=0, partielles=1). Ensuite, la première mesure avec 64 Kio était plus lente qu'avec 4096 octets, ce qui contredisait la logique ; trois nouvelles mesures ont montré que c'était un effet de la première création du fichier destination.

C3 — Métadonnées, permissions et signaux d'alerte

Objectif. Lire les métadonnées d'un fichier et signaler les droits dangereux : écriture pour tous, bits setuid et setgid.

Fichiers. c3_audit.c

Choix documenté. L'analyse utilise lstat, qui ne suit pas les liens symboliques ; le type est déterminé par les macros S_ISREG, S_ISDIR, S_ISLNK, etc. ; les droits sont affichés en octal sur quatre chiffres (st_mode & 07777) pour faire apparaître les bits spéciaux. Alertes sur la sortie d'erreur : setuid, setgid, inscriptible par tous (S_IWOTH, sauf pour un lien dont les droits 0777 ne sont jamais utilisés), propriétaire introuvable (getpwuid), groupe introuvable (getgrgid). Un chemin en erreur est signalé et l'analyse continue. Codes : 0 rien à signaler, 1 usage, 2 au moins un chemin en erreur, 3 au moins une alerte, 4 allocation impossible.

Code source

c3_audit.c

```
sajith@ubuntu24-lab:~/tp2-c$ cat c3_audit.c
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <errno.h>
#include <sys/stat.h>
#include <pwd.h>
#include <grp.h>

static const char *type_de(mode_t m)
{
    if (S_ISREG(m)) return "regulier";
    if (S_ISDIR(m)) return "repertoire";
    if (S_ISLNK(m)) return "lien";
    if (S_ISCHR(m)) return "caractere";
    if (S_ISBLK(m)) return "bloc";
    if (S_ISFIFO(m)) return "tube";
    if (S_ISSOCK(m)) return "socket";
    return "inconnu";
}

int main(int argc, char *argv[])
{
    if (argc < 2) {
        fprintf(stderr, "usage : %s <chemin> [chemin...]\n", argv[0]);
        return 1;
    }

    int n = argc - 1;
    struct stat *infos = malloc((size_t)n * sizeof(struct stat));
    int *valide = malloc((size_t)n * sizeof(int));
    if (infos == NULL || valide == NULL) {
        fprintf(stderr, "erreur : allocation impossible\n");
        free(infos);
        free(valide);
        return 4;
    }

    int erreurs = 0;
    int alertes = 0;

    for (int i = 0; i < n; i++) {
        const char *chemin = argv[i + 1];
        if (lstat(chemin, &infos[i]) != 0) {
```

```
            fprintf(stderr, "erreur : %s : %s\n", chemin, strerror(errno));
            valide[i] = 0;
            erreurs++;
            continue;
        }
        valide[i] = 1;
        printf("%-24s type=%s taille=%lld droits=%04o uid=%u gid=%u\n",
               chemin, type_de(infos[i].st_mode), (long long)infos[i].st_size,
               (unsigned)(infos[i].st_mode & 07777),
               (unsigned)infos[i].st_uid, (unsigned)infos[i].st_gid);
    }

    fflush(stdout);
    fprintf(stderr, "\n");

    for (int i = 0; i < n; i++) {
        if (!valide[i]) {
            continue;
        }
        const char *chemin = argv[i + 1];
        mode_t m = infos[i].st_mode;

        if (m & S_ISUID) {
            fprintf(stderr, "ALERTE %s : droits speciaux actifs (setuid) - a justifier\n",
↪ chemin);
            alertes++;
        }
        if (m & S_ISGID) {
            fprintf(stderr, "ALERTE %s : droits speciaux actifs (setgid) - a justifier\n",
↪ chemin);
            alertes++;
        }
        if (!S_ISLNK(m) && (m & S_IWOTH)) {
            fprintf(stderr, "ALERTE %s : inscriptible par tous\n", chemin);
            alertes++;
        }
        if (getpwuid(infos[i].st_uid) == NULL) {
            fprintf(stderr, "ALERTE %s : proprietaire introuvable (uid=%u)\n", chemin,
↪ (unsigned)infos[i].st_uid);
            alertes++;
        }
        if (getgrgid(infos[i].st_gid) == NULL) {
            fprintf(stderr, "ALERTE %s : groupe introuvable (gid=%u)\n", chemin,
↪ (unsigned)infos[i].st_gid);
            alertes++;
```

```
        }
    }

    fprintf(stderr, "bilan : %d chemin(s), %d erreur(s), %d alerte(s)\n", n, erreurs, alertes);

    free(infos);
    free(valide);

    if (erreurs > 0) {
        return 2;
    }
    if (alertes > 0) {
        return 3;
    }
    return 0;
}
```

Compilation

```
sajith@ubuntu24-lab:~/tp2-c$ gcc -Wall -Wextra -Werror -std=c17 -g -o c3_audit c3_audit.c
[aucune sortie = aucun avertissement]
```

Sorties obtenues

```
Création du jeu de test (commandes à rejouer par le correcteur) :
sajith@ubuntu24-lab:~/tp2-c$ rm -rf ~/tp2-c/jeu && mkdir ~/tp2-c/jeu && cd ~/tp2-c/jeu
sajith@ubuntu24-lab:~/tp2-c$ touch partage.txt && chmod 0666 partage.txt
sajith@ubuntu24-lab:~/tp2-c$ touch prive.txt && chmod 0600 prive.txt
sajith@ubuntu24-lab:~/tp2-c$ ln -s prive.txt lien_prive
sajith@ubuntu24-lab:~/tp2-c$ touch orphelin.txt && sudo chown 4242:4242 orphelin.txt
↪ [uid/gid 4242 : aucun compte sur la machine]
sajith@ubuntu24-lab:~/tp2-c$ ls -l
total 0
lrwxrwxrwx 1 sajith sajith 9 oct.  21 16:19 lien_prive -> prive.txt
-rw-rw-r-- 1   4242   4242 0 oct.  21 16:19 orphelin.txt
-rw-rw-rw- 1 sajith sajith 0 oct.  21 16:19 partage.txt
-rw------- 1 sajith sajith 0 oct.  21 16:19 prive.txt

Fichiers système :
sajith@ubuntu24-lab:~/tp2-c$ ./c3_audit /etc/passwd /usr/bin/passwd ; echo "code=$?"
/etc/passwd              type=regulier taille=1831 droits=0644 uid=0 gid=0
/usr/bin/passwd          type=regulier taille=64152 droits=4755 uid=0 gid=0

ALERTE /usr/bin/passwd : droits speciaux actifs (setuid) - a justifier
bilan : 2 chemin(s), 0 erreur(s), 1 alerte(s)
code=3

Jeu de test (0666, lien symbolique, 0600) — une seule alerte attendue :
sajith@ubuntu24-lab:~/tp2-c$ ./c3_audit jeu/partage.txt jeu/lien_prive jeu/prive.txt ; echo
↪ "code=$?"
jeu/partage.txt          type=regulier taille=0 droits=0666 uid=1000 gid=1000
jeu/lien_prive           type=lien taille=9 droits=0777 uid=1000 gid=1000
jeu/prive.txt            type=regulier taille=0 droits=0600 uid=1000 gid=1000

ALERTE jeu/partage.txt : inscriptible par tous
bilan : 3 chemin(s), 0 erreur(s), 1 alerte(s)
code=3

Propriétaire introuvable, chemin inexistant au milieu de la liste, répertoire :
sajith@ubuntu24-lab:~/tp2-c$ ./c3_audit jeu/orphelin.txt /chemin/inexistant /etc ; echo "code=$?"
jeu/orphelin.txt         type=regulier taille=0 droits=0664 uid=4242 gid=4242
erreur : /chemin/inexistant : No such file or directory
/etc                     type=repertoire taille=4096 droits=0755 uid=0 gid=0

ALERTE jeu/orphelin.txt : proprietaire introuvable (uid=4242)
ALERTE jeu/orphelin.txt : groupe introuvable (gid=4242)
bilan : 3 chemin(s), 1 erreur(s), 2 alerte(s)
code=2
sajith@ubuntu24-lab:~/tp2-c$ ./c3_audit ; echo "code=$?"                                [test
↪  d'erreur volontaire]
usage : ./c3_audit <chemin> [chemin...]
code=1
```

Réponses aux questions

Question 9.1 — Quelle est la différence entre les deux variantes de l'appel d'analyse de métadonnées, et laquelle faut-il employer dans un outil d'audit ? Justifiez par un cas concret.

stat suit les liens symboliques : sur un lien, il décrit le fichier pointé. lstat ne les suit pas : il décrit le lien lui-même. Un outil d'audit doit utiliser lstat. Dans mon jeu de test, lien_prive pointe vers prive.txt : avec stat, l'outil aurait affiché un fichier normal en 0600 et je n'aurais jamais su qu'il y avait un lien. Or un lien posé dans /tmp vers un fichier sensible, c'est justement ce qu'un attaquant prépare pour qu'un programme root écrive au mauvais endroit. En plus, dans un parcours de dossiers, suivre les liens peut faire tourner l'outil en rond.

Question 9.2 — Donnez deux exemples légitimes de programmes porteurs du bit setuid sur un système Linux, et expliquez pour chacun ce que le bit apporte.

Premier exemple : /usr/bin/passwd, en 4755 et appartenant à root (mon outil l'a bien trouvé). Un utilisateur normal doit pouvoir changer son mot de passe, donc écrire dans /etc/shadow, que seul root peut modifier. Le bit setuid fait tourner passwd avec l'identité de root, et le programme ne modifie que la ligne de l'utilisateur qui l'a lancé. Deuxième exemple : /usr/bin/sudo. Il doit lire /etc/sudoers et lancer une commande sous une autre identité, ce que seul root peut faire ; il vérifie

les autorisations avant d'agir. Dans les deux cas, c'est justifié parce que le programme est conçu pour utiliser ces droits de façon contrôlée. Sur un programme mal écrit, le même bit deviendrait une porte d'entrée vers root.

Question 9.3 — Pourquoi un fichier inscriptible par tous placé dans /tmp est-il plus dangereux qu'un fichier inscriptible par tous dans un répertoire privé ?

Pour modifier un fichier, il faut d'abord pouvoir l'atteindre, donc traverser les dossiers de son chemin. Dans un dossier privé en 0700, les autres comptes ne peuvent pas entrer, donc les droits 0666 du fichier ne leur servent à rien. /tmp, au contraire, est ouvert à tout le monde (1777) : n'importe quel compte peut atteindre le fichier et le modifier. Et c'est aussi là que des programmes, même root, déposent leurs fichiers temporaires. Si l'un d'eux lit ou exécute ce fichier modifié, c'est une élévation de privilèges. Le bit sticky de /tmp empêche seulement de supprimer ou renommer le fichier d'un autre, pas de modifier son contenu.

Difficultés rencontrées

Trois points de conception. Un lien symbolique a toujours les droits 0777 : sans exclusion, l'outil aurait levé une fausse alerte « inscriptible par tous » sur lien_prive, alors que le sujet exige exactement les alertes attendues. Créer un fichier dont le propriétaire n'existe pas demande sudo (chown 4242:4242). Enfin, la sortie standard est mise en tampon et la sortie d'erreur non : pour que les alertes apparaissent après les lignes d'analyse, la sortie standard est vidée (fflush) avant le bloc d'alertes.

Partie D — Processus et signaux

D1 — Lancer un programme et récupérer son résultat

Objectif. Créer un processus, exécuter une commande sans passer par un shell, attendre sa fin et interpréter son statut.

Fichiers. d1_lancer.c

Choix documenté. Lancement sans shell : fork, puis execv avec un chemin et le vecteur d'arguments reçu, puis waitpid (rejoué en cas d'EINTR). Les trois issues sont distinguées : terminaison normale (WIFEXITED, code WEXITSTATUS), terminaison par un signal (WIFSIGNALED, numéro WTERMSIG), échec du lancement. Pour détecter l'échec du lancement de façon sûre, un tube relie le fils au parent ; son extrémité d'écriture porte FD_CLOEXEC : si execv réussit, elle est fermée automatiquement et le parent ne lit rien ; si execv échoue, le fils y écrit errno avant de se terminer avec _exit(127). Le parent distingue ainsi un échec de lancement d'une commande qui rendrait elle-même 127. Codes de sortie : celui de la commande, 128 + numéro du signal, 127 pour un échec de lancement, 2 usage, 3 erreur système.

Code source

d1_lancer.c

```
sajith@ubuntu24-lab:~/tp2-c$ cat d1_lancer.c
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <errno.h>
#include <fcntl.h>
#include <unistd.h>
#include <signal.h>
#include <sys/types.h>
#include <sys/wait.h>

int main(int argc, char *argv[])
{
    if (argc < 2) {
        fprintf(stderr, "usage : %s <chemin_absolu> [arguments...]\n", argv[0]);
        return 2;
    }

    int tube[2];
    if (pipe(tube) == -1) {
        fprintf(stderr, "erreur : pipe : %s\n", strerror(errno));
        return 3;
    }
    if (fcntl(tube[1], F_SETFD, FD_CLOEXEC) == -1) {
        fprintf(stderr, "erreur : fcntl : %s\n", strerror(errno));
        close(tube[0]);
        close(tube[1]);
        return 3;
    }

    pid_t pid = fork();
    if (pid == -1) {
        fprintf(stderr, "erreur : fork : %s\n", strerror(errno));
        close(tube[0]);
        close(tube[1]);
        return 3;
    }

    if (pid == 0) {
        close(tube[0]);
        execv(argv[1], &argv[1]);
        int erreur = errno;
        ssize_t r = write(tube[1], &erreur, sizeof(erreur));
        (void)r;
```

```
        _exit(127);
    }

    close(tube[1]);

    int erreur_exec = 0;
    ssize_t lus;
    do {
        lus = read(tube[0], &erreur_exec, sizeof(erreur_exec));
    } while (lus == -1 && errno == EINTR);
    close(tube[0]);

    int statut;
    pid_t r;
    do {
        r = waitpid(pid, &statut, 0);
    } while (r == -1 && errno == EINTR);
    if (r == -1) {
        fprintf(stderr, "erreur : waitpid : %s\n", strerror(errno));
        return 3;
    }

    if (lus == (ssize_t)sizeof(erreur_exec)) {
        fprintf(stderr, "echec du lancement : %s\n", strerror(erreur_exec));
        return 127;
    }

    if (WIFEXITED(statut)) {
        printf("commande terminee normalement, code %d\n", WEXITSTATUS(statut));
        return WEXITSTATUS(statut);
    }
    if (WIFSIGNALED(statut)) {
        int sig = WTERMSIG(statut);
        printf("commande terminee par le signal %d (%s)\n", sig, strsignal(sig));
        return 128 + sig;
    }

    fprintf(stderr, "erreur : terminaison inattendue\n");
    return 3;
}
```

Compilation

```
sajith@ubuntu24-lab:~/tp2-c$ gcc -Wall -Wextra -Werror -std=c17 -g -o d1_lancer d1_lancer.c
[aucune sortie = aucun avertissement]
```

Sorties obtenues

```
Les quatre cas de la démonstration :
sajith@ubuntu24-lab:~/tp2-c$ ./d1_lancer /bin/true ; echo "code=$?"
commande terminee normalement, code 0
code=0
sajith@ubuntu24-lab:~/tp2-c$ ./d1_lancer /bin/false ; echo "code=$?"
commande terminee normalement, code 1
code=1
sajith@ubuntu24-lab:~/tp2-c$ ./d1_lancer /bin/ls -d /tmp ; echo "code=$?"
/tmp
commande terminee normalement, code 0
code=0
sajith@ubuntu24-lab:~/tp2-c$ ./d1_lancer /chemin/inexistant ; echo "code=$?"            [test
↪ d'erreur volontaire : échec du lancement]
echec du lancement : No such file or directory
code=127

Terminaison provoquée par un signal :
sajith@ubuntu24-lab:~/tp2-c$ ./d1_lancer /bin/sleep 60 &
[1] 2029
sajith@ubuntu24-lab:~/tp2-c$ ps -o pid,ppid,stat,cmd --ppid 2029                        [le fils
↪  sleep, dont le parent est d1_lancer]
    PID    PPID STAT CMD
   2031    2029 S    /bin/sleep 60
sajith@ubuntu24-lab:~/tp2-c$ kill -TERM $(pgrep -P 2029)
↪ [équivalent de kill -TERM 2031 depuis un second terminal]
commande terminee par le signal 15 (Terminated)
[1]+  Exit 143                ./d1_lancer /bin/sleep 60
sajith@ubuntu24-lab:~/tp2-c$ echo "code=$?"
code=143

Absence d'injection de commande (pas de shell) :
sajith@ubuntu24-lab:~/tp2-c$ ./d1_lancer /bin/echo "fichier.txt; rm -rf /tmp/test" ; echo
↪ "code=$?"
fichier.txt; rm -rf /tmp/test
commande terminee normalement, code 0
code=0

Vérifications :
sajith@ubuntu24-lab:~/tp2-c$ ps -eo stat,pid,cmd | awk '$1 ~ /^Z/' | wc -l            [processus
↪  zombies après exécution]
0
sajith@ubuntu24-lab:~/tp2-c$ valgrind --track-fds=yes ./d1_lancer /bin/true 2>&1 | grep -E "FILE
↪  DESCRIPTORS|commande"
commande terminee normalement, code 0
==2039== FILE DESCRIPTORS: 3 open (3 std) at exit.
```

```
sajith@ubuntu24-lab:~/tp2-c$ grep -c "system" d1_lancer.c                               [aucun
↪ appel à system()]
0
```

Réponses aux questions

Question 10.1 — Expliquez, avec un exemple concret, comment un argument contenant un point-virgule peut faire exécuter une commande non prévue si le programme passe par un shell.

Imaginons un outil qui affiche un fichier choisi par l'utilisateur en construisant une commande : snprintf(cmd, sizeof(cmd), "cat %s", nom) puis system(cmd). Si l'utilisateur donne comme nom « rapport.txt; rm -rf ~ », la commande devient « cat rapport.txt; rm -rf ~ ». Le shell voit le point-virgule comme un séparateur : il affiche le fichier, puis efface le dossier personnel, avec les droits du programme. Si l'outil tourne en root, l'attaquant peut lancer n'importe quoi en root. Avec execv, il n'y a pas de shell : le texte est passé tel quel comme un seul argument. Mon test le prouve : ./d1_lancer /bin/echo "fichier.txt; rm -rf /tmp/test" affiche juste le texte, point-virgule compris, sans rien exécuter d'autre.

Question 10.2 — Comment distingue-t-on, à partir du statut rendu par l'attente, une terminaison normale d'une terminaison provoquée par un signal ? Nommez les macros utilisées.

waitpid donne un entier de statut qu'il ne faut pas lire directement : on utilise les macros de <sys/wait.h>. WIFEXITED(statut) est vrai si le fils s'est terminé normalement, et WEXITSTATUS(statut) donne alors son code de sortie (0 pour /bin/true, 1 pour /bin/false).

WIFSIGNALED(statut) est vrai si le fils a été tué par un signal, et WTERMSIG(statut) donne le numéro du signal : 15 (SIGTERM) dans ma démonstration. Comme le shell, mon programme rend alors 128 + le numéro du signal, soit 143.

Question 10.3 — Qu'est-ce qu'un processus zombie, et pourquoi le programme que vous venez d'écrire n'en produit-il pas ?

Un zombie, c'est un processus qui est terminé mais dont le parent n'a pas encore récupéré le statut de fin. Il ne fait plus rien et n'utilise plus de mémoire, mais il reste dans la table des processus (état Z dans ps) en attendant que le parent appelle wait ou waitpid. Si un programme lance plein de fils sans jamais les attendre, les zombies s'accumulent et peuvent finir par remplir la table. Mon programme appelle toujours waitpid sur son fils avant de se terminer, même quand le lancement a échoué. Ma vérification après exécution le confirme : 0 processus en état Z.

Difficultés rencontrées

La principale difficulté est de distinguer l'échec du lancement d'une commande qui se termine normalement avec le code 127 : se fier au seul code de sortie du fils serait ambigu. Le tube marqué FD_CLOEXEC lève cette ambiguïté. Dans le fils, après un execv raté, la sortie se fait par _exit et non par exit, pour ne pas vider une seconde fois les tampons d'entrée-sortie hérités du parent.

D2 — Arrêt propre sur signal

Objectif. Intercepter une demande d'arrêt sans se bloquer et terminer proprement, sans attente active.

Fichiers. d2_veille.c

Choix documenté. Le gestionnaire, installé avec sigaction pour SIGINT, SIGTERM et SIGUSR1, ne fait qu'incrémenter un compteur et enregistrer la demande d'arrêt dans des variables volatile sig_atomic_t ; les messages et le bilan sont produits par la boucle principale. La temporisation de 1 s utilise nanosleep : si elle est interrompue (EINTR), elle est reprise avec le temps restant, sauf si un arrêt a été demandé. SIGUSR1 est seulement compté, pour démontrer cette reprise. SIGPIPE est ignoré (SIG_IGN) et le retour de printf et fflush est vérifié : une sortie fermée arrête le programme avec un message et le code 3. La durée réelle est mesurée avec CLOCK_MONOTONIC pour comparer le compteur au temps écoulé. Codes : 0 arrêt propre, 2 erreur système, 3 sortie standard fermée.

Code source

d2_veille.c

```
sajith@ubuntu24-lab:~/tp2-c$ cat d2_veille.c
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <string.h>
#include <errno.h>
#include <signal.h>
#include <time.h>
#include <unistd.h>

static volatile sig_atomic_t arret_demande = 0;
static volatile sig_atomic_t signaux_recus = 0;

static void gestionnaire(int sig)
{
    signaux_recus++;
    if (sig == SIGINT || sig == SIGTERM) {
        arret_demande = sig;
    }
}

int main(void)
{
    struct sigaction sa;
    memset(&sa, 0, sizeof(sa));
    sa.sa_handler = gestionnaire;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;

    if (sigaction(SIGINT, &sa, NULL) == -1 || sigaction(SIGTERM, &sa, NULL) == -1 ||
        sigaction(SIGUSR1, &sa, NULL) == -1) {
        fprintf(stderr, "erreur : sigaction : %s\n", strerror(errno));
        return 2;
    }

    signal(SIGPIPE, SIG_IGN);

    struct timespec debut, fin;
    clock_gettime(CLOCK_MONOTONIC, &debut);

    if (printf("veille demarree (pid %ld) - Ctrl-C pour arreter\n", (long)getpid()) < 0 ||
↪ fflush(stdout) == EOF) {
        fprintf(stderr, "erreur : ecriture impossible sur la sortie standard\n");
        return 3;
    }
```

```
    long ticks = 0;
    int sortie_fermee = 0;

    while (!arret_demande) {
        struct timespec reste = {1, 0};
        while (nanosleep(&reste, &reste) == -1) {
            if (errno != EINTR) {
                fprintf(stderr, "erreur : nanosleep : %s\n", strerror(errno));
                return 2;
            }
            if (arret_demande) {
                break;
            }
        }
        if (arret_demande) {
            break;
        }

        if (printf("tick %ld\n", ticks + 1) < 0 || fflush(stdout) == EOF) {
            sortie_fermee = 1;
            break;
        }
        ticks++;
    }

    clock_gettime(CLOCK_MONOTONIC, &fin);
    double duree = (double)(fin.tv_sec - debut.tv_sec) + (double)(fin.tv_nsec - debut.tv_nsec) /
↪  1e9;

    if (sortie_fermee) {
        fprintf(stderr, "sortie standard fermee (%s) : arret apres %ld ticks affiches\n",
↪ strerror(errno), ticks);
        fprintf(stderr, "signaux recus : %d, duree reelle : %.2f s\n", (int)signaux_recus, duree);
        return 3;
    }

    printf("arret demande : signal %d recu\n", (int)arret_demande);
    printf("veille arretee proprement apres %ld ticks (signaux recus : %d, duree reelle : %.2f
↪ s)\n",
           ticks, (int)signaux_recus, duree);
    return 0;
}
```

Compilation

```
sajith@ubuntu24-lab:~/tp2-c$ gcc -Wall -Wextra -Werror -std=c17 -g -o d2_veille d2_veille.c
[aucune sortie = aucun avertissement]
```

Sorties obtenues

```
Test 1 — arrêt par Ctrl-C (interactif) :
sajith@ubuntu24-lab:~/tp2-c$ ./d2_veille ; echo "code=$?"
veille demarree (pid 2051) - Ctrl-C pour arreter
tick 1
tick 2
tick 3
tick 4
tick 5
tick 6
tick 7
tick 8
tick 9
tick 10
tick 11
tick 12
tick 13
tick 14
tick 15
^Carret demande : signal 2 recu
veille arretee proprement apres 15 ticks (signaux recus : 1, duree reelle : 15.43 s)
code=0

Test 2 — arrêt par kill depuis le shell, avec deux signaux SIGUSR1 pendant la veille :
sajith@ubuntu24-lab:~/tp2-c$ ./d2_veille &
[1] 2052
veille demarree (pid 2052) - Ctrl-C pour arreter
sajith@ubuntu24-lab:~/tp2-c$ sleep 3.5 ; kill -USR1 2052 ; sleep 0.2 ; kill -USR1 2052   [deux
↪ temporisations interrompues puis reprises]
tick 1
tick 2
tick 3
tick 4
tick 5
sajith@ubuntu24-lab:~/tp2-c$ ps -o pid,stat,%cpu,time,cmd -p 2052
↪ [consommation au repos]
    PID STAT %CPU     TIME CMD
   2052 S     0.0 00:00:00 ./d2_veille
sajith@ubuntu24-lab:~/tp2-c$ kill -TERM 2052
arret demande : signal 15 recu
veille arretee proprement apres 5 ticks (signaux recus : 3, duree reelle : 5.24 s)
[1]+  Done                    ./d2_veille
sajith@ubuntu24-lab:~/tp2-c$ echo "code=$?"
code=0

Test 3 — écriture vers une sortie fermée (SIGPIPE ignoré, retour des écritures vérifié) :
```

```
sajith@ubuntu24-lab:~/tp2-c$ ./d2_veille | head -3 ; echo "codes : d2_veille=${PIPESTATUS[0]}
↪ head=${PIPESTATUS[1]}"
veille demarree (pid 2057) - Ctrl-C pour arreter
tick 1
tick 2
sortie standard fermee (Broken pipe) : arret apres 2 ticks affiches
signaux recus : 0, duree reelle : 3.00 s
codes : d2_veille=3 head=0
```

Réponses aux questions

Question 11.1 — Pourquoi la fonction de traitement du signal ne doit-elle faire qu'un geste minimal ? Citez deux fonctions qu'il ne faut pas y appeler et expliquez le risque.

Un signal peut arriver à n'importe quel moment, comme une sonnette, y compris au milieu d'un printf ou d'un malloc du programme principal. Le gestionnaire s'exécute alors « par-dessus » ce qui était en cours. S'il appelle lui-même printf, il peut tomber sur le verrou interne de printf déjà pris par le programme principal, et attendre pour toujours : le programme est bloqué. S'il appelle malloc ou free pendant un malloc interrompu, il peut corrompre la mémoire. Ces fonctions ne sont pas « async-signal-safe ». C'est pour ça que mon gestionnaire se contente de noter le signal dans une variable volatile sig_atomic_t, et que c'est la boucle principale qui affiche les messages et fait le bilan.

Question 11.2 — Que se passe-t-il par défaut lorsqu'un programme écrit vers une sortie dont l'autre extrémité est fermée ? Comment l'éviter, et dans quel cas cela vous est-il déjà arrivé au TP 1 ?

Par défaut, le noyau envoie le signal SIGPIPE au programme, et ce signal le tue immédiatement, sans aucun message : le programme disparaît en plein travail. Pour l'éviter, on ignore SIGPIPE avec signal(SIGPIPE, SIG_IGN). L'écriture échoue alors simplement avec l'erreur EPIPE, et il faut vérifier le retour de chaque écriture pour s'arrêter proprement. Mon test 3 le montre : avec ./d2_veille | head -3, head s'arrête après 3 lignes, et mon programme affiche « sortie standard fermee (Broken pipe) » avec le code 3 au lieu de mourir en silence. Au TP 1, j'ai souvent utilisé ce genre de commande, comme hostnamectl | head -5 ou systemctl list-units ... | head -30 : si la commande de gauche avait encore à écrire quand head s'arrêtait, elle était tuée sans rien afficher. Pour un simple affichage ce n'est pas grave, mais pour un outil qui doit produire un résultat complet, si.

Question 11.3 — Qu'est-ce qu'une attente active, et pourquoi votre programme ne doit-il pas en faire ? Quelle observation le prouve dans votre compte rendu ?

Une attente active, c'est une boucle qui vérifie sans arrêt une condition (« la seconde est passée ? ») au lieu de dormir. Le programme ne va pas plus vite, mais il occupe un cœur du processeur à 100 % pour rien, au détriment des autres programmes. Le mien utilise nanosleep : le noyau le met en sommeil et ne le réveille qu'au bout de la seconde ou quand un signal arrive. La preuve est dans mon test 2 : ps affiche l'état S (en sommeil), 0.0 % de processeur, et 00:00:00 de temps processeur cumulé après plus de 5 secondes de fonctionnement.

Difficultés rencontrées

Deux points de conception. Une temporisation interrompue ne doit pas être comptée comme une seconde complète : nanosleep rend le temps restant, réutilisé pour reprendre l'attente ; le test 2 le vérifie : 5 ticks pour 5,24 s réelles malgré deux signaux SIGUSR1. Ensuite, le tick ne doit être compté qu'une fois réellement écrit : dans le test 3, l'écriture du troisième tick échoue et le bilan annonce bien 2 ticks affichés.

Partie E — Synthèse évaluée

E1 — Synthèse évaluée : l'outil d'audit d'arborescence

Objectif. Assembler tout ce qui précède : parcours récursif robuste, règles de sécurité, journal, résumé et arrêt propre.

Fichiers. e1_audit.h, e1_regles.c, e1_journal.c, e1_parcours.c, e1_main.c, Makefile (exécutable e1_audit_arbo)

Choix documenté. Découpage (comme en A3) : e1_regles.c applique les règles de C3 (setuid, setgid, inscriptible par tous hors liens, propriétaire ou groupe introuvable) sans rien afficher ; e1_journal.c ouvre le journal en mode ajout (O_APPEND | O_CLOEXEC, droits 0640) et écrit une ligne horodatée par événement, marquée du numéro de processus ; e1_parcours.c parcourt récursivement avec opendir, readdir et lstat ; e1_main.c gère les arguments, les signaux (D2), le résumé et le relevé des descripteurs (/proc/self/fd). Précautions du parcours : les entrées « . » et « .. » sont filtrées ; les liens symboliques sont analysés avec lstat mais jamais suivis ; la profondeur est bornée à 64 niveaux ; chaque chemin est construit avec snprintf dans un tampon de 4096 octets dont le retour est vérifié (chemin trop long : entrée signalée et ignorée) ; toute erreur d'accès est signalée sur la sortie d'erreur et dans le journal, puis le parcours continue. Arrêt propre : SIGINT et SIGTERM positionnent un indicateur testé avant chaque entrée ; le parcours remonte en fermant chaque répertoire ouvert, le journal reçoit une ligne ARRET puis FIN et il est fermé ; le résumé est marqué PARTIEL. Codes : 0 audit complet sans erreur, 1 usage, 2 erreur du journal ou système, 3 audit complet avec erreurs d'accès, 4 audit interrompu. Le journal /var/log/audit-arbo.log n'existait pas avant la première exécution sous sudo, qui l'a créé.

Code source

e1_audit.h

```
sajith@ubuntu24-lab:~/tp2-c/e1$ cat e1_audit.h
#ifndef E1_AUDIT_H
#define E1_AUDIT_H

#include <stdio.h>
#include <signal.h>
#include <sys/stat.h>

#define MAX_PROFONDEUR 64
#define TAILLE_CHEMIN 4096

enum {
    REGLE_SETUID = 1,
    REGLE_SETGID = 2,
    REGLE_ECRITURE_TOUS = 4,
    REGLE_SANS_PROPRIETAIRE = 8,
    REGLE_SANS_GROUPE = 16
};

struct bilan {
    long analyses;
    long en_alerte;
    long alertes;
    long speciaux;
    long ecriture_tous;
    long sans_proprietaire;
    long erreurs;
    long profondeur_max_atteinte;
};

extern volatile sig_atomic_t arret_demande;

int analyser(const struct stat *infos);

FILE *journal_ouvrir(const char *chemin);
void journal_ecrire(FILE *journal, const char *format, ...);
int journal_fermer(FILE *journal);

void parcourir(const char *chemin, int profondeur, FILE *journal, struct bilan *b);

int compter_descripteurs(void);

#endif
```

e1_regles.c

```
sajith@ubuntu24-lab:~/tp2-c/e1$ cat e1_regles.c
#define _POSIX_C_SOURCE 200809L
#include <pwd.h>
#include <grp.h>
#include "e1_audit.h"

int analyser(const struct stat *infos)
{
    int resultat = 0;
    mode_t m = infos->st_mode;

    if (m & S_ISUID) {
        resultat |= REGLE_SETUID;
    }
    if (m & S_ISGID) {
        resultat |= REGLE_SETGID;
    }
    if (!S_ISLNK(m) && (m & S_IWOTH)) {
        resultat |= REGLE_ECRITURE_TOUS;
    }
    if (getpwuid(infos->st_uid) == NULL) {
        resultat |= REGLE_SANS_PROPRIETAIRE;
    }
    if (getgrgid(infos->st_gid) == NULL) {
        resultat |= REGLE_SANS_GROUPE;
    }
    return resultat;
}
```

e1_journal.c

```
sajith@ubuntu24-lab:~/tp2-c/e1$ cat e1_journal.c
#define _POSIX_C_SOURCE 200809L
#include <stdarg.h>
#include <time.h>
#include <unistd.h>
#include <fcntl.h>
#include "e1_audit.h"

FILE *journal_ouvrir(const char *chemin)
{
    int fd = open(chemin, O_WRONLY | O_CREAT | O_APPEND | O_CLOEXEC, 0640);
    if (fd == -1) {
        return NULL;
    }
    FILE *f = fdopen(fd, "a");
    if (f == NULL) {
        close(fd);
    }
    return f;
}

void journal_ecrire(FILE *journal, const char *format, ...)
{
    char horodatage[32];
    time_t maintenant = time(NULL);
    struct tm local;

    if (localtime_r(&maintenant, &local) == NULL ||
        strftime(horodatage, sizeof(horodatage), "%Y-%m-%dT%H:%M:%S%z", &local) == 0) {
        horodatage[0] = '\0';
    }

    fprintf(journal, "%s [%ld] ", horodatage, (long)getpid());
    va_list args;
    va_start(args, format);
    vfprintf(journal, format, args);
    va_end(args);
    fputc('\n', journal);
}

int journal_fermer(FILE *journal)
{
    return fclose(journal);
}
```

e1_parcours.c

```
sajith@ubuntu24-lab:~/tp2-c/e1$ cat e1_parcours.c
#define _POSIX_C_SOURCE 200809L
#include <string.h>
#include <errno.h>
#include <dirent.h>
#include "e1_audit.h"

static void signaler(FILE *journal, struct bilan *b, const char *chemin, const struct stat
↪ *infos, int regles)
{
    unsigned droits = (unsigned)(infos->st_mode & 07777);
    unsigned uid = (unsigned)infos->st_uid;
    unsigned gid = (unsigned)infos->st_gid;

    b->en_alerte++;

    if (regles & REGLE_SETUID) {
        journal_ecrire(journal, "ALERTE setuid %s (droits=%04o uid=%u gid=%u)", chemin, droits,
↪ uid, gid);
        b->alertes++;
    }
    if (regles & REGLE_SETGID) {
        journal_ecrire(journal, "ALERTE setgid %s (droits=%04o uid=%u gid=%u)", chemin, droits,
↪ uid, gid);
        b->alertes++;
    }
    if (regles & (REGLE_SETUID | REGLE_SETGID)) {
        b->speciaux++;
    }
    if (regles & REGLE_ECRITURE_TOUS) {
        journal_ecrire(journal, "ALERTE inscriptible-par-tous %s (droits=%04o uid=%u gid=%u)",
↪ chemin, droits, uid, gid);
        b->alertes++;
        b->ecriture_tous++;
    }
    if (regles & REGLE_SANS_PROPRIETAIRE) {
        journal_ecrire(journal, "ALERTE proprietaire-introuvable %s (uid=%u)", chemin, uid);
        b->alertes++;
    }
    if (regles & REGLE_SANS_GROUPE) {
        journal_ecrire(journal, "ALERTE groupe-introuvable %s (gid=%u)", chemin, gid);
        b->alertes++;
    }
    if (regles & (REGLE_SANS_PROPRIETAIRE | REGLE_SANS_GROUPE)) {
        b->sans_proprietaire++;
```

```
    }
}

static void erreur(FILE *journal, struct bilan *b, const char *chemin, const char *quoi, int code)
{
    b->erreurs++;
    fprintf(stderr, "erreur : %s : %s : %s\n", chemin, quoi, strerror(code));
    journal_ecrire(journal, "ERREUR %s %s : %s", quoi, chemin, strerror(code));
}

void parcourir(const char *chemin, int profondeur, FILE *journal, struct bilan *b)
{
    if (arret_demande) {
        return;
    }

    struct stat infos;
    if (lstat(chemin, &infos) != 0) {
        erreur(journal, b, chemin, "lstat", errno);
        return;
    }

    b->analyses++;
    int regles = analyser(&infos);
    if (regles != 0) {
        signaler(journal, b, chemin, &infos, regles);
    }

    if (!S_ISDIR(infos.st_mode)) {
        return;
    }

    if (profondeur >= MAX_PROFONDEUR) {
        b->profondeur_max_atteinte++;
        erreur(journal, b, chemin, "profondeur maximale atteinte", ELOOP);
        return;
    }

    DIR *rep = opendir(chemin);
    if (rep == NULL) {
        erreur(journal, b, chemin, "opendir", errno);
        return;
    }

    for (;;) {
```

```
        if (arret_demande) {
            break;
        }

        errno = 0;
        struct dirent *entree = readdir(rep);
        if (entree == NULL) {
            if (errno != 0) {
                erreur(journal, b, chemin, "readdir", errno);
            }
            break;
        }

        if (strcmp(entree->d_name, ".") == 0 || strcmp(entree->d_name, "..") == 0) {
            continue;
        }

        char sous_chemin[TAILLE_CHEMIN];
        const char *separateur = (chemin[strlen(chemin) - 1] == '/') ? "" : "/";
        int n = snprintf(sous_chemin, sizeof(sous_chemin), "%s%s%s", chemin, separateur,
↪ entree->d_name);
        if (n < 0 || (size_t)n >= sizeof(sous_chemin)) {
            erreur(journal, b, chemin, "chemin trop long, entree ignoree", ENAMETOOLONG);
            continue;
        }

        parcourir(sous_chemin, profondeur + 1, journal, b);
    }

    if (closedir(rep) != 0) {
        erreur(journal, b, chemin, "closedir", errno);
    }
}
```

e1_main.c

```
sajith@ubuntu24-lab:~/tp2-c/e1$ cat e1_main.c
#define _POSIX_C_SOURCE 200809L
#include <string.h>
#include <errno.h>
#include <dirent.h>
#include "e1_audit.h"

volatile sig_atomic_t arret_demande = 0;

static void gestionnaire(int sig)
{
    arret_demande = sig;
}

int compter_descripteurs(void)
{
    DIR *rep = opendir("/proc/self/fd");
    if (rep == NULL) {
        return -1;
    }
    int n = 0;
    struct dirent *e;
    while ((e = readdir(rep)) != NULL) {
        if (strcmp(e->d_name, ".") != 0 && strcmp(e->d_name, "..") != 0) {
            n++;
        }
    }
    closedir(rep);
    return n - 1;
}

int main(int argc, char *argv[])
{
    const char *chemin_journal = "audit-arbo.log";
    int premier = 1;

    if (argc >= 3 && strcmp(argv[1], "--journal") == 0) {
        chemin_journal = argv[2];
        premier = 3;
    }
    if (premier >= argc) {
        fprintf(stderr, "usage : %s [--journal fichier] <repertoire> [repertoire...]\n", argv[0]);
        return 1;
    }
```

```
    struct sigaction sa;
    memset(&sa, 0, sizeof(sa));
    sa.sa_handler = gestionnaire;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;
    if (sigaction(SIGINT, &sa, NULL) == -1 || sigaction(SIGTERM, &sa, NULL) == -1) {
        fprintf(stderr, "erreur : sigaction : %s\n", strerror(errno));
        return 2;
    }
    signal(SIGPIPE, SIG_IGN);

    int fd_debut = compter_descripteurs();

    FILE *journal = journal_ouvrir(chemin_journal);
    if (journal == NULL) {
        fprintf(stderr, "erreur : ouverture du journal %s : %s\n", chemin_journal,
↪ strerror(errno));
        return 2;
    }

    struct bilan b;
    memset(&b, 0, sizeof(b));

    printf("analyse de");
    for (int i = premier; i < argc; i++) {
        printf(" %s", argv[i]);
    }
    printf("\n");
    journal_ecrire(journal, "DEBUT audit (%d arborescence(s))", argc - premier);

    for (int i = premier; i < argc && !arret_demande; i++) {
        parcourir(argv[i], 0, journal, &b);
    }

    if (arret_demande) {
        journal_ecrire(journal, "ARRET demande par le signal %d : resultat partiel",
↪ (int)arret_demande);
    }
    journal_ecrire(journal, "FIN audit : analyses=%ld en_alerte=%ld alertes=%ld erreurs=%ld",
                   b.analyses, b.en_alerte, b.alertes, b.erreurs);

    int fd_avant_fermeture = compter_descripteurs();
    int erreur_journal = journal_fermer(journal) != 0;
    int fd_fin = compter_descripteurs();
```

```
    if (arret_demande) {
        printf("ARRET DEMANDE (signal %d) : RESUME PARTIEL\n", (int)arret_demande);
    }
    printf("fichiers analyses       : %ld\n", b.analyses);
    printf("fichiers en alerte      : %ld\n", b.en_alerte);
    printf("  setuid ou setgid      : %ld\n", b.speciaux);
    printf("  inscriptibles par tous: %ld\n", b.ecriture_tous);
    printf("  sans proprietaire     : %ld\n", b.sans_proprietaire);
    printf("lignes d'alerte         : %ld\n", b.alertes);
    printf("erreurs d'acces         : %ld\n", b.erreurs);
    printf("descripteurs ouverts    : debut=%d, avec journal=%d, fin=%d\n", fd_debut,
↪ fd_avant_fermeture, fd_fin);

    if (erreur_journal) {
        fprintf(stderr, "erreur : fermeture du journal %s\n", chemin_journal);
        return 2;
    }
    printf("rapport ecrit dans %s\n", chemin_journal);

    if (arret_demande) {
        return 4;
    }
    return b.erreurs > 0 ? 3 : 0;
}
```

Makefile

```
sajith@ubuntu24-lab:~/tp2-c/e1$ cat Makefile
CC = gcc
CFLAGS = -Wall -Wextra -Werror -std=c17 -g
CIBLE = e1_audit_arbo
OBJETS = e1_main.o e1_parcours.o e1_regles.o e1_journal.o

all: $(CIBLE)

$(CIBLE): $(OBJETS)
    $(CC) $(CFLAGS) -o $(CIBLE) $(OBJETS)

%.o: %.c e1_audit.h
    $(CC) $(CFLAGS) -c $<

asan:
    $(CC) $(CFLAGS) -fsanitize=address,undefined -o $(CIBLE)_asan e1_main.c e1_parcours.c
↪ e1_regles.c e1_journal.c

clean:
    rm -f $(OBJETS) $(CIBLE) $(CIBLE)_asan

.PHONY: all asan clean
```

Compilation

```
sajith@ubuntu24-lab:~/tp2-c/e1$ make clean && make && make asan
rm -f e1_main.o e1_parcours.o e1_regles.o e1_journal.o e1_audit_arbo e1_audit_arbo_asan
gcc -Wall -Wextra -Werror -std=c17 -g -c e1_main.c
gcc -Wall -Wextra -Werror -std=c17 -g -c e1_parcours.c
gcc -Wall -Wextra -Werror -std=c17 -g -c e1_regles.c
gcc -Wall -Wextra -Werror -std=c17 -g -c e1_journal.c
gcc -Wall -Wextra -Werror -std=c17 -g -o e1_audit_arbo e1_main.o e1_parcours.o e1_regles.o
↪ e1_journal.o
gcc -Wall -Wextra -Werror -std=c17 -g -fsanitize=address,undefined -o e1_audit_arbo_asan
↪ e1_main.c e1_parcours.c e1_regles.c e1_journal.c
[aucun avertissement]
```

Sorties obtenues

```
Création de l'arborescence piégée (commandes à rejouer par le correcteur) :
sajith@ubuntu24-lab:~/tp2-c/e1$ rm -rf ~/tp2-c/piege; mkdir -p ~/tp2-c/piege/sous/profond; cd
↪ ~/tp2-c/piege
sajith@ubuntu24-lab:~/tp2-c/e1$ ln -s .. sous/boucle_parent                          [lien qui
↪ remonte vers le parent]
sajith@ubuntu24-lab:~/tp2-c/e1$ ln -s ~/tp2-c/piege boucle_racine                    [lien vers
↪ la racine de l'arborescence]
sajith@ubuntu24-lab:~/tp2-c/e1$ touch partage.txt && chmod 0666 partage.txt          [fichier
↪ inscriptible par tous]
sajith@ubuntu24-lab:~/tp2-c/e1$ cp /bin/true faux_setuid && chmod 4755 faux_setuid   [programme
↪ setuid factice]
sajith@ubuntu24-lab:~/tp2-c/e1$ mkdir secret && touch secret/cache.txt && chmod 000 secret
↪ [répertoire sans droit de lecture]
sajith@ubuntu24-lab:~/tp2-c/e1$ touch orphelin.txt && sudo chown 4242:4242 orphelin.txt
↪ [propriétaire et groupe inexistants]
sajith@ubuntu24-lab:~/tp2-c/e1$ touch sous/profond/normal.txt
sajith@ubuntu24-lab:~/tp2-c/e1$ ls -la . sous
.:
total 44
drwxrwxr-x 4 sajith sajith  4096 oct.  21 16:32 .
drwxrwxr-x 5 sajith sajith  4096 oct.  21 16:32 ..
lrwxrwxrwx 1 sajith sajith    24 oct.  21 16:32 boucle_racine -> /home/sajith/tp2-c/piege
-rwsr-xr-x 1 sajith sajith 26936 oct.  21 16:32 faux_setuid
-rw-rw-r-- 1   4242   4242     0 oct.  21 16:32 orphelin.txt
-rw-rw-rw- 1 sajith sajith     0 oct.  21 16:32 partage.txt
d--------- 2 sajith sajith  4096 oct.  21 16:32 secret
drwxrwxr-x 3 sajith sajith  4096 oct.  21 16:32 sous

sous:
total 12
drwxrwxr-x 3 sajith sajith 4096 oct.  21 16:32 .
drwxrwxr-x 4 sajith sajith 4096 oct.  21 16:32 ..
lrwxrwxrwx 1 sajith sajith    2 oct.  21 16:32 boucle_parent -> ..
drwxrwxr-x 2 sajith sajith 4096 oct.  21 16:32 profond

Audit de l'arborescence piégée (compte ordinaire) :
sajith@ubuntu24-lab:~/tp2-c/e1$ ./e1_audit_arbo --journal piege.log ~/tp2-c/piege ; echo "code=$?"
analyse de /home/sajith/tp2-c/piege
erreur : /home/sajith/tp2-c/piege/secret : opendir : Permission denied
fichiers analyses       : 10
fichiers en alerte      : 3
  setuid ou setgid      : 1
  inscriptibles par tous: 1
  sans proprietaire     : 1
lignes d'alerte         : 4
```

```
erreurs d'acces         : 1
descripteurs ouverts    : debut=3, avec journal=4, fin=3
rapport ecrit dans piege.log
code=3
sajith@ubuntu24-lab:~/tp2-c/e1$ cat piege.log
2026-10-21T16:32:58+0200 [2118] DEBUT audit (1 arborescence(s))
2026-10-21T16:32:58+0200 [2118] ALERTE inscriptible-par-tous
↪ /home/sajith/tp2-c/piege/partage.txt (droits=0666 uid=1000 gid=1000)
2026-10-21T16:32:58+0200 [2118] ERREUR opendir /home/sajith/tp2-c/piege/secret : Permission denied
2026-10-21T16:32:58+0200 [2118] ALERTE proprietaire-introuvable
↪ /home/sajith/tp2-c/piege/orphelin.txt (uid=4242)
2026-10-21T16:32:58+0200 [2118] ALERTE groupe-introuvable /home/sajith/tp2-c/piege/orphelin.txt
↪ (gid=4242)
2026-10-21T16:32:58+0200 [2118] ALERTE setuid /home/sajith/tp2-c/piege/faux_setuid (droits=4755
↪ uid=1000 gid=1000)
2026-10-21T16:32:58+0200 [2118] FIN audit : analyses=10 en_alerte=3 alertes=4 erreurs=1

Analyse mémoire (valgrind) :
sajith@ubuntu24-lab:~/tp2-c/e1$ valgrind --leak-check=full --track-fds=yes ./e1_audit_arbo
↪ --journal v.log ~/tp2-c/piege 2>&1 | grep -E "FILE DESCRIPTORS|in use at exit|All heap|ERROR
↪ SUMMARY"
==2120== FILE DESCRIPTORS: 3 open (3 std) at exit.
==2120==     in use at exit: 5,542 bytes in 13 blocks
==2120== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
sajith@ubuntu24-lab:~/tp2-c/e1$ valgrind --leak-check=full --show-leak-kinds=all ./e1_audit_arbo
↪  --journal v.log ~/tp2-c/piege 2>&1 | grep -E "definitely lost|indirectly lost|possibly
↪ lost|still reachable|getpwuid|getgrgid|nss" | head -20
==2125== 42 bytes in 1 blocks are still reachable in loss record 1 of 8
==2125== 42 bytes in 1 blocks are still reachable in loss record 2 of 8
==2125== 66 bytes in 2 blocks are still reachable in loss record 3 of 8
==2125== 66 bytes in 2 blocks are still reachable in loss record 4 of 8
==2125== 100 bytes in 1 blocks are still reachable in loss record 5 of 8
==2125==    by 0x4E7FAD3: _nss_systemd_is_blocked (in /usr/lib/x86_64-linux-
↪ gnu/libnss_systemd.so.2)
==2125==    by 0x4E81C17: _nss_systemd_getpwuid_r (in /usr/lib/x86_64-linux-
↪ gnu/libnss_systemd.so.2)
==2125==    by 0x49C2E93: getpwuid_r@@GLIBC_2.2.5 (getXXbyYY_r.c:273)
==2125==    by 0x49C2CA6: getpwuid (getXXbyYY.c:140)
==2125== 1,260 bytes in 1 blocks are still reachable in loss record 6 of 8
==2125== 1,464 bytes in 3 blocks are still reachable in loss record 7 of 8
==2125==    by 0x49B3D5E: module_load (nss_module.c:187)
==2125== 2,502 bytes in 2 blocks are still reachable in loss record 8 of 8
==2125==    definitely lost: 0 bytes in 0 blocks
==2125==    indirectly lost: 0 bytes in 0 blocks
==2125==      possibly lost: 0 bytes in 0 blocks
```

```
==2125==    still reachable: 5,542 bytes in 13 blocks

Assainisseur d'adresses et de comportements indéfinis :
sajith@ubuntu24-lab:~/tp2-c/e1$ make asan
sajith@ubuntu24-lab:~/tp2-c/e1$ ./e1_audit_arbo_asan --journal a.log ~/tp2-c/piege > /dev/null ;
↪  echo "code=$?"
erreur : /home/sajith/tp2-c/piege/secret : opendir : Permission denied
code=3
[aucun rapport de l'assainisseur]

Audit de /etc sous délégation (journal dans /var/log, créé à la première exécution) :
sajith@ubuntu24-lab:~/tp2-c/e1$ sudo ./e1_audit_arbo --journal /var/log/audit-arbo.log /etc ;
↪ echo "code=$?"
analyse de /etc
fichiers analyses       : 1721
fichiers en alerte      : 0
  setuid ou setgid      : 0
  inscriptibles par tous: 0
  sans proprietaire     : 0
lignes d'alerte         : 0
erreurs d'acces         : 0
descripteurs ouverts    : debut=3, avec journal=4, fin=3
rapport ecrit dans /var/log/audit-arbo.log
code=0

Audit de /etc et /usr/bin (comparaison compteurs / lignes du journal) :
sajith@ubuntu24-lab:~/tp2-c/e1$ sudo ./e1_audit_arbo --journal /var/log/audit-arbo.log /etc
↪ /usr/bin ; echo "code=$?"
analyse de /etc /usr/bin
fichiers analyses       : 2851
fichiers en alerte      : 14
  setuid ou setgid      : 14
  inscriptibles par tous: 0
  sans proprietaire     : 0
lignes d'alerte         : 14
erreurs d'acces         : 0
descripteurs ouverts    : debut=3, avec journal=4, fin=3
rapport ecrit dans /var/log/audit-arbo.log
code=0
sajith@ubuntu24-lab:~/tp2-c/e1$ sudo cat /var/log/audit-arbo.log | grep -c " ALERTE "     [le
↪ journal ne contient que ces deux exécutions ; la première n'a aucune alerte]
14
sajith@ubuntu24-lab:~/tp2-c/e1$ sudo cat /var/log/audit-arbo.log
2026-10-21T16:34:48+0200 [2133] DEBUT audit (1 arborescence(s))
2026-10-21T16:34:48+0200 [2133] FIN audit : analyses=1721 en_alerte=0 alertes=0 erreurs=0
```

```
2026-10-21T16:36:25+0200 [2156] DEBUT audit (2 arborescence(s))
2026-10-21T16:36:25+0200 [2156] ALERTE setuid /usr/bin/fusermount3 (droits=4755 uid=0 gid=0)
2026-10-21T16:36:25+0200 [2156] ALERTE setuid /usr/bin/su (droits=4755 uid=0 gid=0)
2026-10-21T16:36:25+0200 [2156] ALERTE setuid /usr/bin/chfn (droits=4755 uid=0 gid=0)
2026-10-21T16:36:25+0200 [2156] ALERTE setuid /usr/bin/newgrp (droits=4755 uid=0 gid=0)
2026-10-21T16:36:25+0200 [2156] ALERTE setgid /usr/bin/ssh-agent (droits=2755 uid=0 gid=103)
2026-10-21T16:36:25+0200 [2156] ALERTE setuid /usr/bin/mount (droits=4755 uid=0 gid=0)
2026-10-21T16:36:25+0200 [2156] ALERTE setuid /usr/bin/gpasswd (droits=4755 uid=0 gid=0)
2026-10-21T16:36:25+0200 [2156] ALERTE setuid /usr/bin/sudo (droits=4755 uid=0 gid=0)
2026-10-21T16:36:25+0200 [2156] ALERTE setgid /usr/bin/crontab (droits=2755 uid=0 gid=990)
2026-10-21T16:36:25+0200 [2156] ALERTE setuid /usr/bin/umount (droits=4755 uid=0 gid=0)
2026-10-21T16:36:25+0200 [2156] ALERTE setuid /usr/bin/passwd (droits=4755 uid=0 gid=0)
2026-10-21T16:36:25+0200 [2156] ALERTE setuid /usr/bin/chsh (droits=4755 uid=0 gid=0)
2026-10-21T16:36:25+0200 [2156] ALERTE setgid /usr/bin/expiry (droits=2755 uid=0 gid=42)
2026-10-21T16:36:25+0200 [2156] ALERTE setgid /usr/bin/chage (droits=2755 uid=0 gid=42)
2026-10-21T16:36:25+0200 [2156] FIN audit : analyses=2851 en_alerte=14 alertes=14 erreurs=0

Audit de /etc sous un compte ordinaire (erreurs de permission signalées, pas contournées) :
sajith@ubuntu24-lab:~/tp2-c/e1$ ./e1_audit_arbo --journal etc_user.log /etc 2>&1 | tail -12
erreur : /etc/ssl/private : opendir : Permission denied
erreur : /etc/polkit-1/rules.d : opendir : Permission denied
analyse de /etc
fichiers analyses       : 1718
fichiers en alerte      : 0
  setuid ou setgid      : 0
  inscriptibles par tous: 0
  sans proprietaire     : 0
lignes d'alerte         : 0
erreurs d'acces         : 7
descripteurs ouverts    : debut=3, avec journal=4, fin=3
rapport ecrit dans etc_user.log

Arrêt demandé en cours de parcours (SIGINT envoyé après 0,1 s) :
sajith@ubuntu24-lab:~/tp2-c/e1$ timeout -s INT 0.1 ./e1_audit_arbo --journal interruption.log
↪ /usr ; echo "code=$?"
analyse de /usr
ARRET DEMANDE (signal 2) : RESUME PARTIEL
fichiers analyses       : 1790
fichiers en alerte      : 0
  setuid ou setgid      : 0
  inscriptibles par tous: 0
  sans proprietaire     : 0
lignes d'alerte         : 0
erreurs d'acces         : 0
descripteurs ouverts    : debut=3, avec journal=4, fin=3
```

```
rapport ecrit dans interruption.log
code=124                                                [code rendu par timeout, pas par
↪ l'outil]
sajith@ubuntu24-lab:~/tp2-c/e1$ tail -3 interruption.log
2026-10-21T16:34:48+0200 [2146] DEBUT audit (1 arborescence(s))
2026-10-21T16:34:48+0200 [2146] ARRET demande par le signal 2 : resultat partiel
2026-10-21T16:34:48+0200 [2146] FIN audit : analyses=1790 en_alerte=0 alertes=0 erreurs=0
sajith@ubuntu24-lab:~/tp2-c/e1$ timeout --preserve-status -s INT 0.1 ./e1_audit_arbo --journal
↪ interruption2.log /usr ; echo "code=$?"
analyse de /usr
ARRET DEMANDE (signal 2) : RESUME PARTIEL
fichiers analyses       : 3952
fichiers en alerte      : 15
  setuid ou setgid      : 15
  inscriptibles par tous: 0
  sans proprietaire     : 0
lignes d'alerte         : 15
erreurs d'acces         : 0
descripteurs ouverts    : debut=3, avec journal=4, fin=3
rapport ecrit dans interruption2.log
code=4                                                  [code rendu par l'outil : arrêt
↪ demandé]
```

Réponses aux questions

Question 12.1 — Décrivez les trois précautions que votre parcours prend pour ne pas boucler ni s'arrêter, et ce qui se produirait sans chacune.

Je vois le parcours comme la visite d'un bâtiment, pièce par pièce. Première précaution : ignorer les entrées « . » et « .. », qui ramènent dans la même pièce ou dans la pièce précédente. Sans ça, l'outil tournerait en boucle jusqu'à planter. Deuxième précaution : ne pas suivre les liens symboliques (lstat), parce qu'un lien peut renvoyer vers une pièce déjà visitée. Dans mon arborescence piégée, boucle_parent pointe vers .. et boucle_racine vers la racine de l'audit : en les suivant, l'outil ne s'arrêterait jamais. La limite de 64 niveaux de profondeur ajoute une sécurité en plus. Troisième précaution : continuer après une erreur d'accès. Le dossier secret en 0000 fait échouer opendir ; je note l'erreur dans le journal et je passe à la suite. Sans ça, le premier dossier fermé à clé arrêterait tout l'audit. Mon test le confirme : 10 entrées analysées, 4 alertes, 1 erreur, et aucune boucle.

Question 12.2 — Pourquoi borner la longueur des chemins construits ? Que se passerait-il si un chemin dépassait la taille de votre tampon ?

Chaque chemin est construit en collant le chemin du dossier, un « / » et le nom de l'entrée. Ces noms viennent du système de fichiers, donc éventuellement d'un utilisateur malveillant qui peut créer des dossiers très profonds ou des noms très longs. Si on copiait sans limite (strcpy, strcat, sprintf), un chemin plus long que mon tampon de 4096 octets déborderait sur la pile, exactement comme en B2 : plantage, ou pire, détournement d'un outil qui tourne en root. Dans mon programme, snprintf ne peut jamais écrire au-delà du tampon. S'il rend une longueur supérieure ou égale à la taille, ça veut dire que le chemin a été coupé : je signale l'entrée comme « chemin trop long » et je ne l'utilise pas, parce qu'un chemin coupé désignerait un autre fichier.

Question 12.3 — En quoi un outil d'audit qui s'interrompt sur une erreur d'accès est-il plus dangereux qu'un outil qui n'existe pas ?

Sans outil, l'administrateur sait que l'audit n'a pas été fait et reste prudent. Un outil qui s'arrête au premier dossier illisible donne un rapport incomplet qui ressemble à un rapport complet, par exemple « 0 alerte ». L'administrateur pense alors que la machine est saine alors que ce n'est pas vrai : c'est un faux négatif, le pire résultat possible pour un contrôle de sécurité. Un attaquant pourrait même s'en servir exprès : il suffit de créer un dossier illisible placé avant ses fichiers piégés pour rendre l'audit aveugle sur tout le reste. Mon outil continue après chaque erreur, la note dans le journal, la compte dans le résumé (7 erreurs pour /etc sans sudo) et rend un code à part (3). On voit donc tout de suite que l'audit est incomplet.

Difficultés rencontrées

Le premier rapport de valgrind indiquait 5 542 octets encore utilisés à la sortie. Une seconde analyse (--show-leak-kinds=all) a montré qu'il n'y a aucune fuite (definitely, indirectly et possibly lost à 0) : ces blocs « still reachable » sont alloués par getpwuid dans la bibliothèque libnss_systemd et par le chargement des modules NSS, que la bibliothèque C conserve jusqu'à la fin du processus. Deuxième point : pour compter les lignes du journal, la commande sudo wc -l < /var/log/audit-arbo.log a échoué (« Permission denied ») car la redirection < est faite par le shell de l'utilisateur, pas par sudo ; le comptage a donc porté sur tout le journal, qui ne contenait que les deux exécutions, la première sans alerte : 14 lignes pour 14 alertes annoncées. Enfin, le code 124 affiché après timeout est celui de timeout lui-même ; avec --preserve-status, on obtient le vrai code de l'outil, 4 (audit interrompu).

Annexe 1 — Journal de bord

Heure	Action	Message d'erreur brut	Correction
17/10 15:29	Connexion SSH depuis le poste hôte vers 192.168.113.128	ssh: connect to host 192.168.113.128 port 22: Connection timed out	VM encore en cours de démarrage. Vérifications : ssh écoute sur :22, ufw autorise OpenSSH, ping OK. Nouvelle tentative réussie.
19/10 19:26	A1 : test d'un argument inconnu	aucun message affiché, code=2	Ajout des messages sur la sortie d'erreur (fprintf(stderr, ...)) et du cas « trop d'arguments » avec le code 1.
20/10 17:55	A3 : création du Makefile par copier-coller	risque « missing separator » : tabulations remplacées par des espaces	sed -i 's/^ /\t/' Makefile, puis vérification avec cat -A (^I).
20/10 18:01	A4 : démonstration de la division entière avec 5 5 5	moyenne=5.00 aussi avec la version fausse	Choix d'un jeu dont la somme n'est pas divisible (12 7 30 -4 et 1 2) pour faire apparaître l'écart.
21/10 15:49	Reprise sur le PC portable après déplacement de la VM	ssh vers l'ancienne adresse impossible (nouveau réseau NAT)	Nouvelle adresse relevée avec ip -br a : 192.168.105.128.
21/10 16:13	C2 : mesures avec quatre tailles de bloc	bloc 65536 : temps reel=0,376 s, plus lent que bloc 4096 (0,056 s)	Trois nouvelles mesures : 0,041 à 0,068 s. Biais de la première copie (création du fichier destination).
21/10 16:33	E1 : analyse mémoire avec valgrind	in use at exit: 5,542 bytes in 13 blocks	--show-leak-kinds=all : definitely lost 0 ; blocs still reachable alloués par getpwuid (libnss_systemd).
21/10 16:36	E1 : comptage des lignes du journal système	-bash: /var/log/audit-arbo.log: Permission denied	La redirection est faite par le shell utilisateur, pas par sudo ; comptage sur le journal complet (deux exécutions).

Annexe 2 — Grille d'auto-évaluation

Je sais faire, seul, sans aide	Acquis / à revoir	Preuve dans mon compte rendu
Compiler avec les options du module et obtenir zéro avertissement	☐ acquis ☐ à revoir	Tous les exercices (compilation sans avertissement)
Valider une entrée et rendre un code de sortie distinct par cause	☐ acquis ☐ à revoir	A1, A2 (codes distincts par cause)
Découper un programme en modules et l'assembler par Makefile	☐ acquis ☐ à revoir	A3, E1 (Makefile, en-têtes)
Allouer, utiliser et libérer de la mémoire sans fuite	☐ acquis ☐ à revoir	B1 (valgrind : no leaks are possible)
Reconnaître et corriger un débordement de tampon	☐ acquis ☐ à revoir	B2 (assainisseur, b2_sur.c)
Lire un fichier par blocs bornés et distinguer fin de fichier et erreur	☐ acquis ☐ à revoir	C1 (fgets, ferror, /proc/self/mem)
Utiliser les appels système bruts et gérer les écritures partielles	☐ acquis ☐ à revoir	C2 (EINTR, écriture partielle, mesures)
Lire les métadonnées d'un fichier et repérer un droit dangereux	☐ acquis ☐ à revoir	C3, E1 (lstat, alertes)
Créer un processus et interpréter sa terminaison	☐ acquis ☐ à revoir	D1 (fork, execv, waitpid)
Intercepter un signal et terminer proprement	☐ acquis ☐ à revoir	D2 (sigaction, arrêt propre)
Diagnostiquer avec les outils d'analyse et de traçage	☐ acquis ☐ à revoir	B1, B2, C2, E1 (valgrind, ASan, strace)
Documenter un livrable pour qu'un tiers le reproduise	☐ acquis ☐ à revoir	Compte rendu complet, jeux de test reproductibles